KENNESAW STATE
UNIVERSITY

CS 4277
Convolutional Neural Network

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

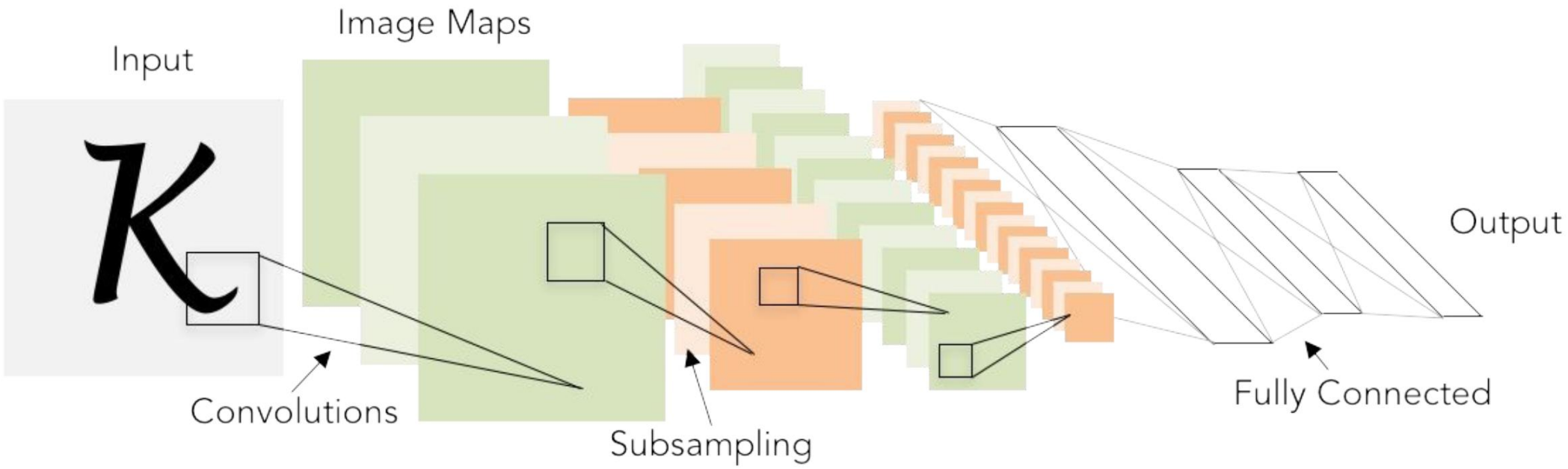
College of Computing and Software Engineering

Learning Objectives

- Describe the operation of convolution layers
- Describe the difference between fully connected layers and convolution layers
- Apply the common layers and architectures used in convolutional neural networks

Background

Convolutional Neural Networks (ConvNets)



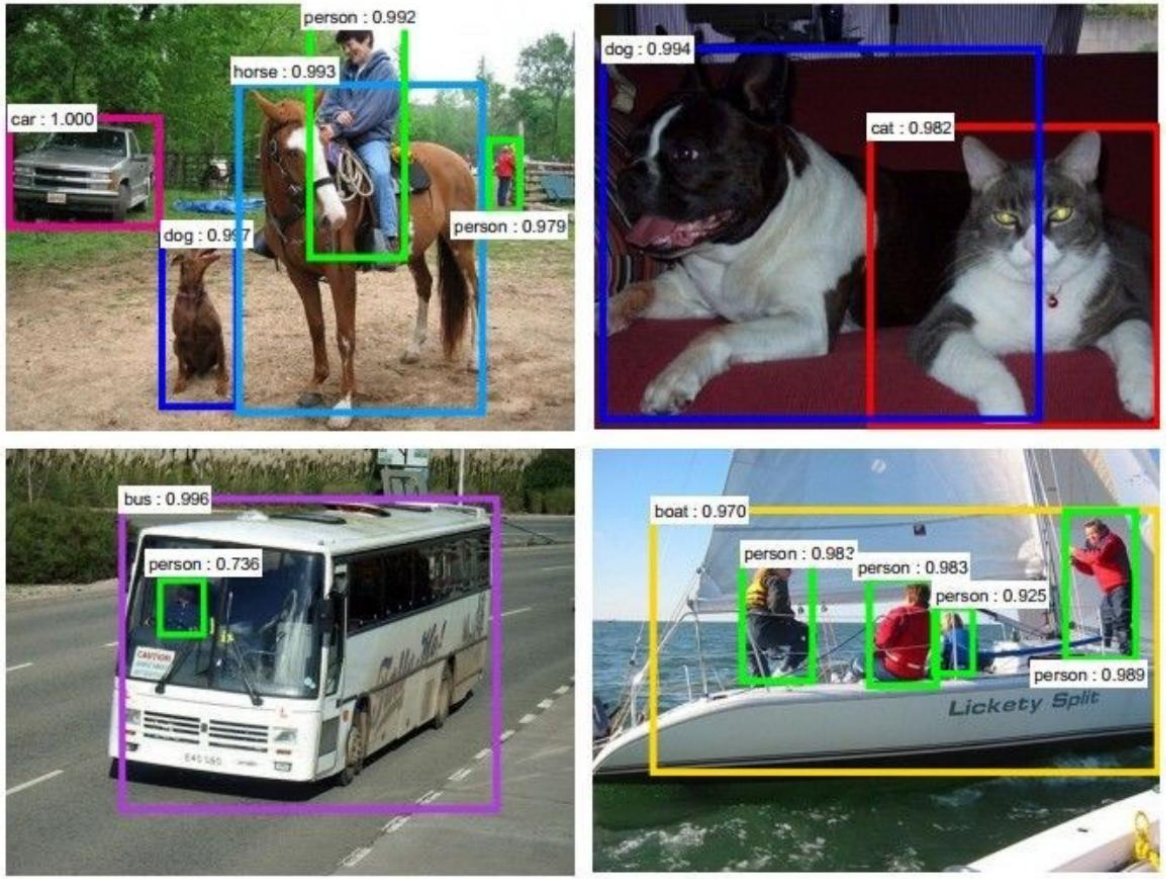
ConvNets are everywhere

Classification

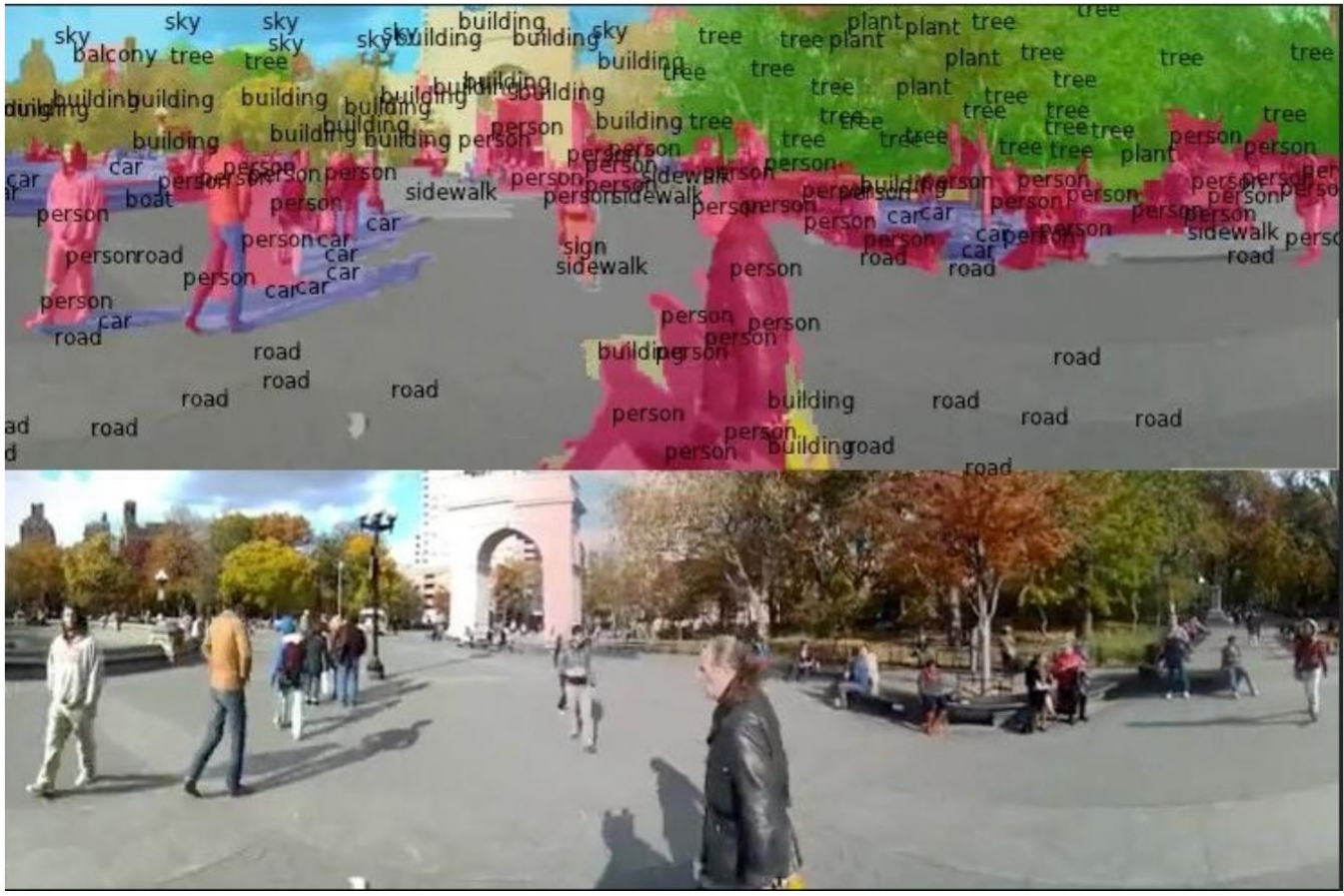
Retrieval

ConvNets are everywhere

Detection

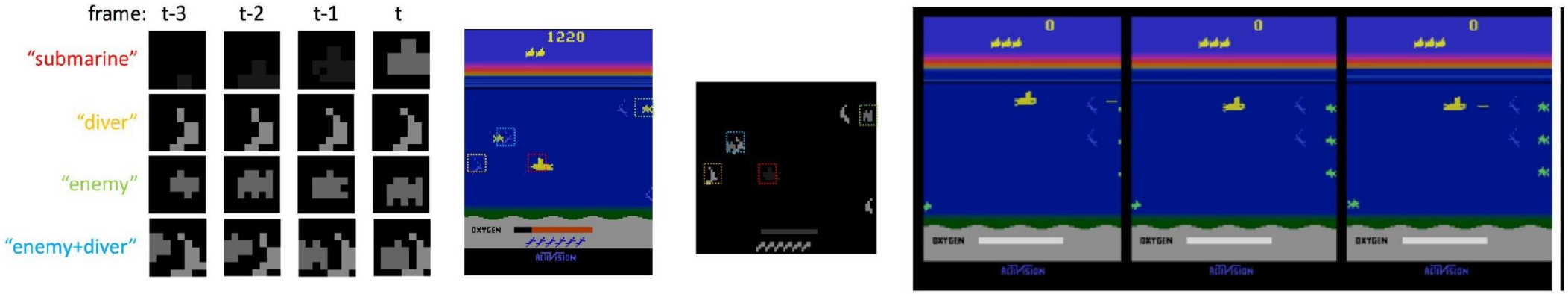


Segmentation



ConvNets are everywhere

[Toshev, Szegedy 2014]



[Guo et al. 2014]

Challenges in Image Classification

Challenges in Image Classification

- Regular Neural Nets don’t scale well to full images
- Neural Networks
  - Extremely high dimensionality!
  - 256x256 image has already 65,536 * 3 dimensions
  - One hidden layer with 500 hidden units require 65,536 * 3 * 500 connections (98 Million parameters)
- Huge number of parameters would quickly lead to overfitting.

Challenges in Image Classification

- Local invariance
  - Objects in natural images have local invariance characteristics
  - Operations such as scaling, translation, and rotation do not affect its semantic information.
  - The fully connected feedforward network is difficult to extract these local invariant features

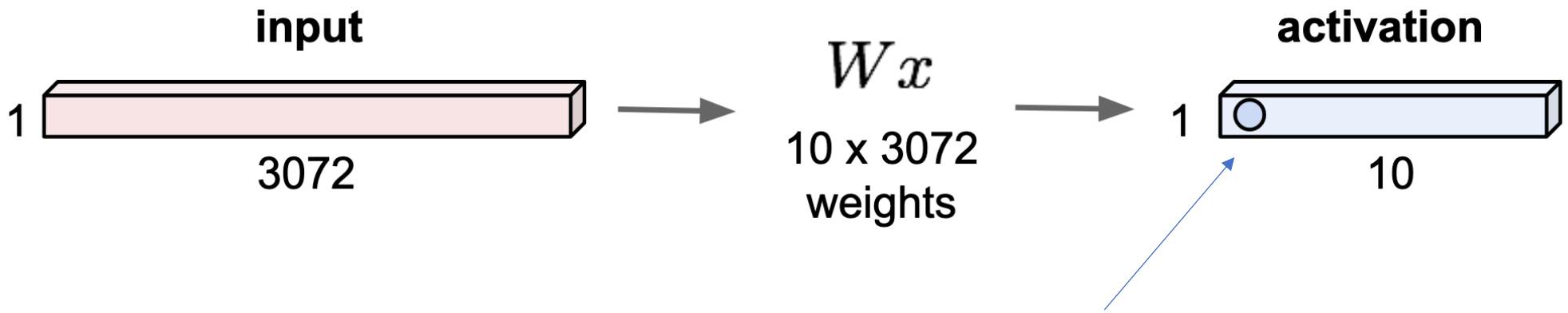
Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN)

- Convolutional Neural Networks (CNN) are very similar to ordinary Neural Networks
- CNN made up of neurons that have learnable weights and biases
- Each neuron receives some inputs, performs a dot product and optionally follows it with a non-linearity
- The whole network still expresses a single differentiable score function:
  - from the raw image pixels on one end to class scores at the other

Challenges in Image Classification

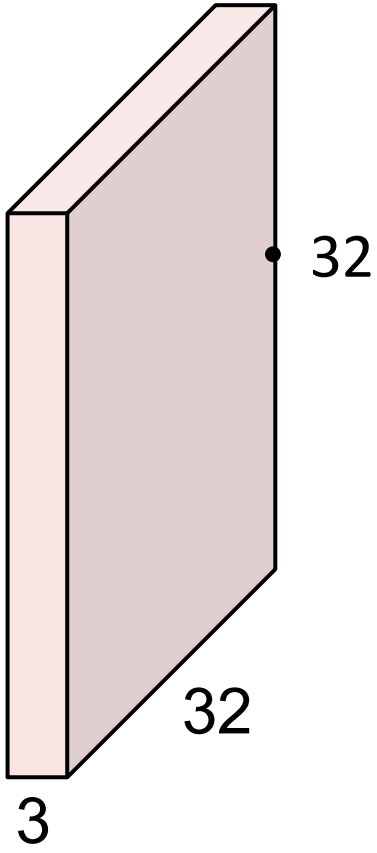
- 32x32x3 image -> stretch to 3072 x 1
- Fully connected layer



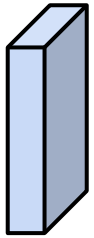
1 number:
the result of taking a dot product between a row of W and the input (a 3072-dimensional dot product)

Convolution Layer

- 32x32x3 image -> preserve spatial structure



- 5x5x3 filter

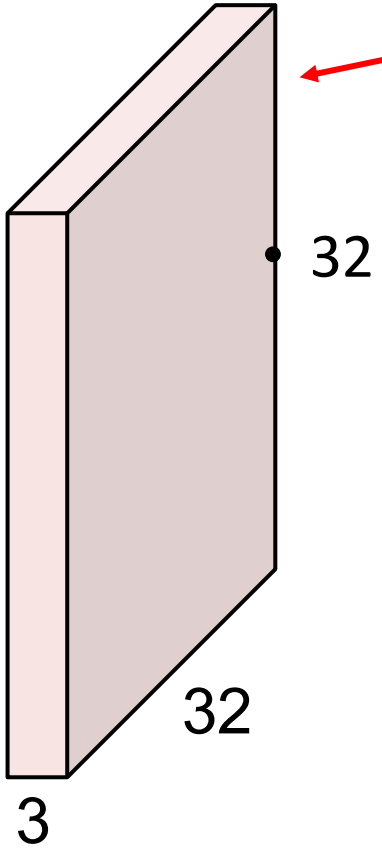


- **Convolve** the filter with the image
- i.e. “slide over the image spatially, computing dot products”

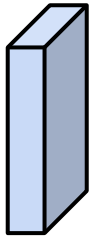
Convolution Layer

Filters always extend the full depth of the input volume

- 32x32x3

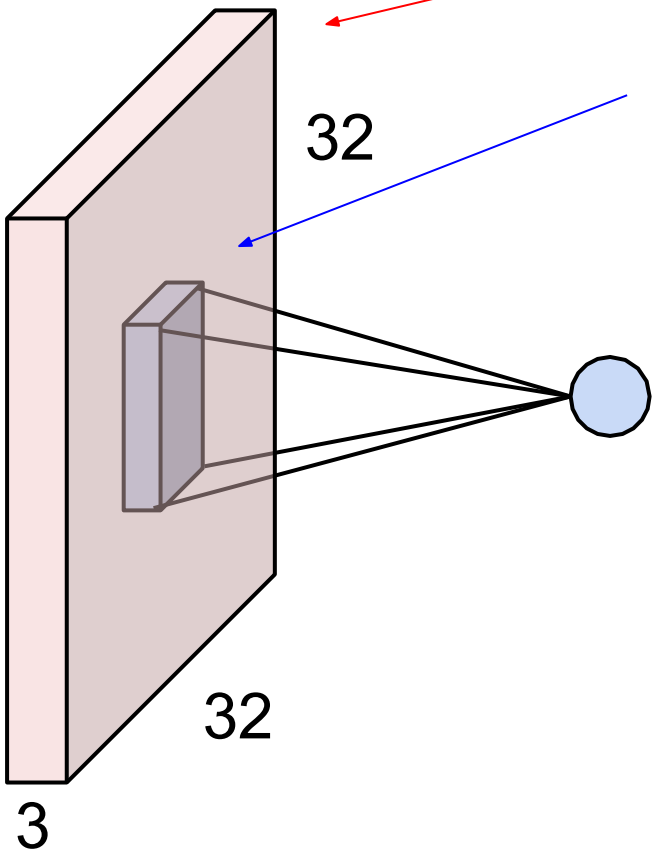


- 5x5x3 filter



- **Convolve** the filter with the image
- i.e. “slide over the image spatially, computing dot products”

Convolution Layer



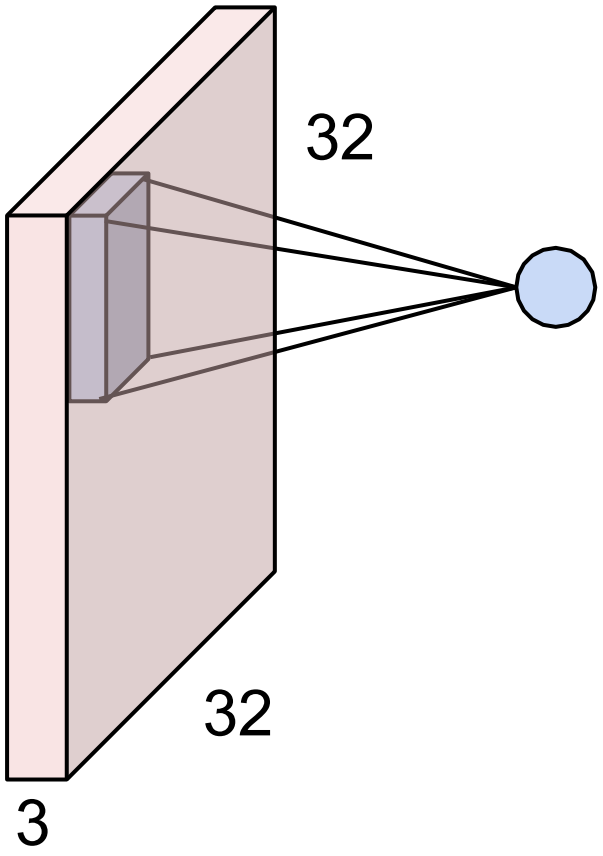
32x32x3 image

5x5x3 filter w

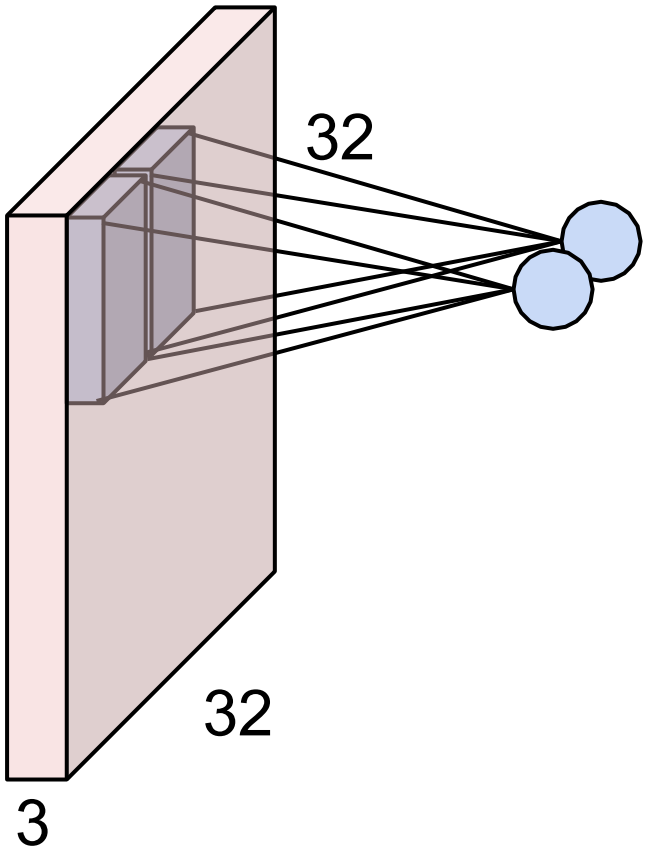
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

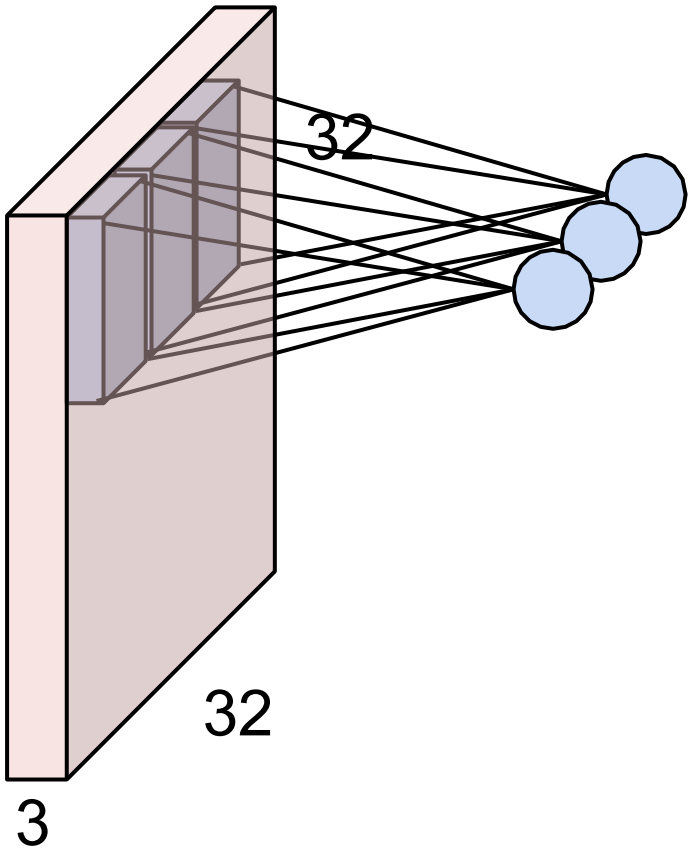
Convolution Layer



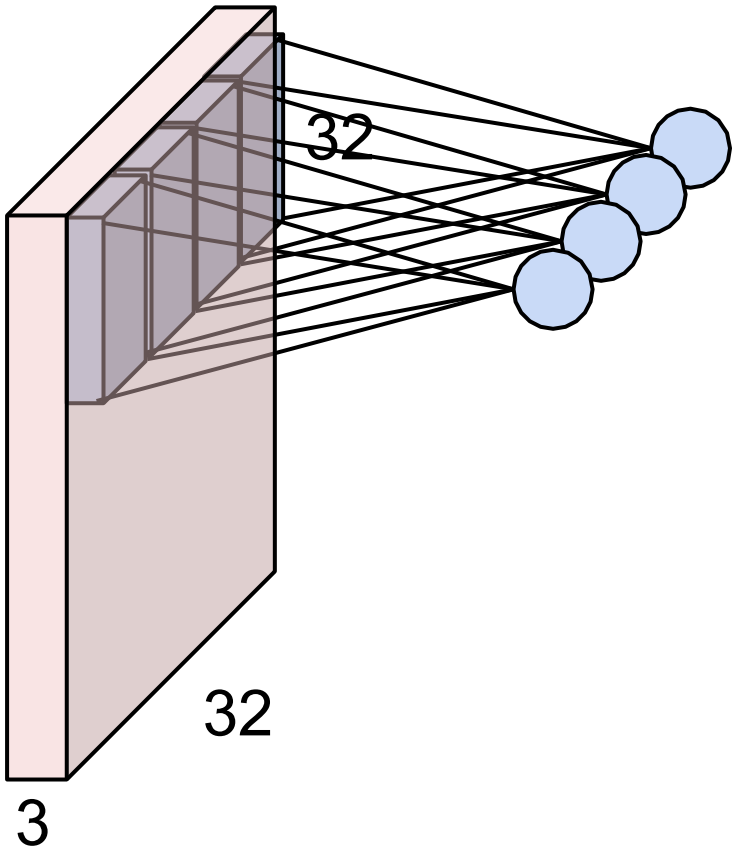
Convolution Layer



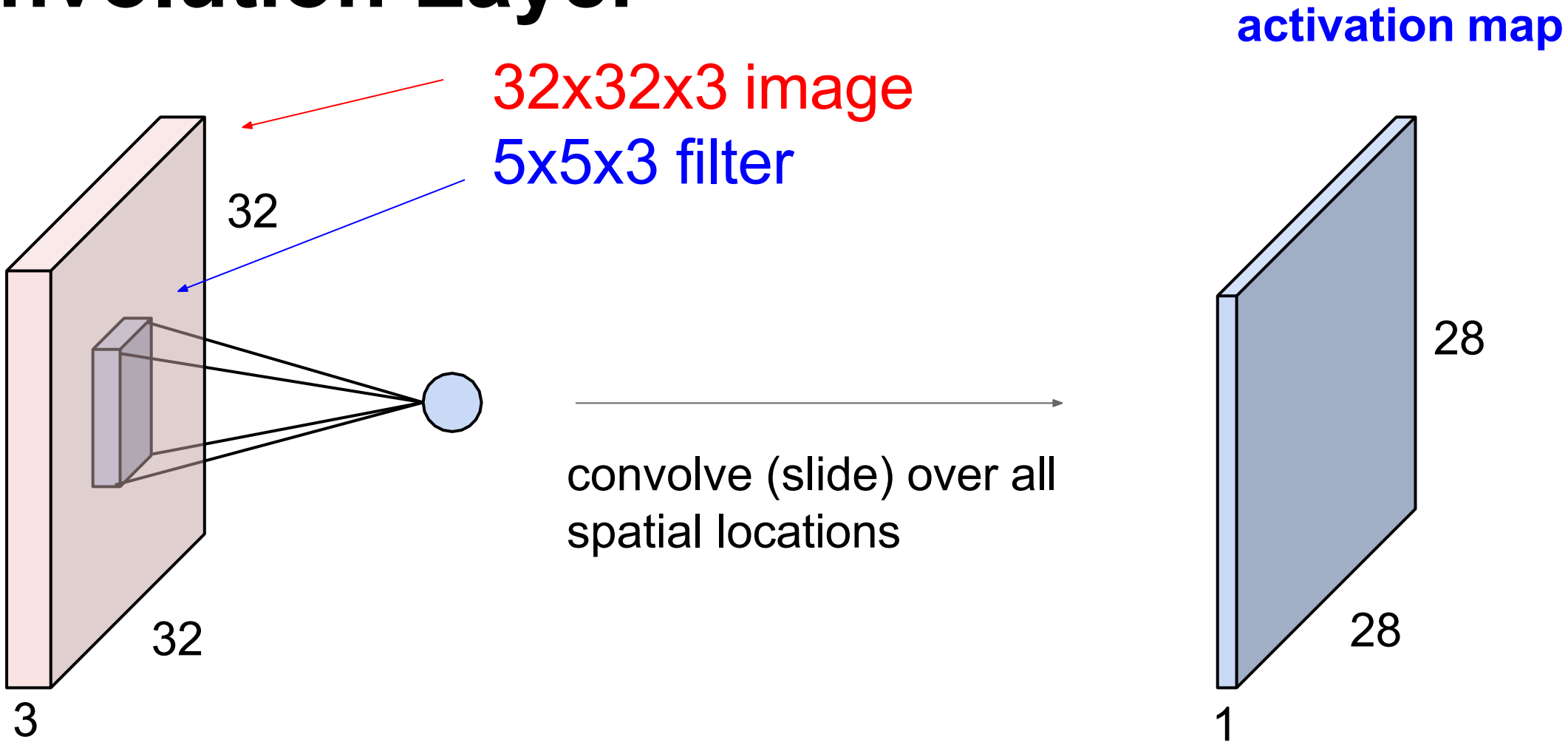
Convolution Layer



Convolution Layer

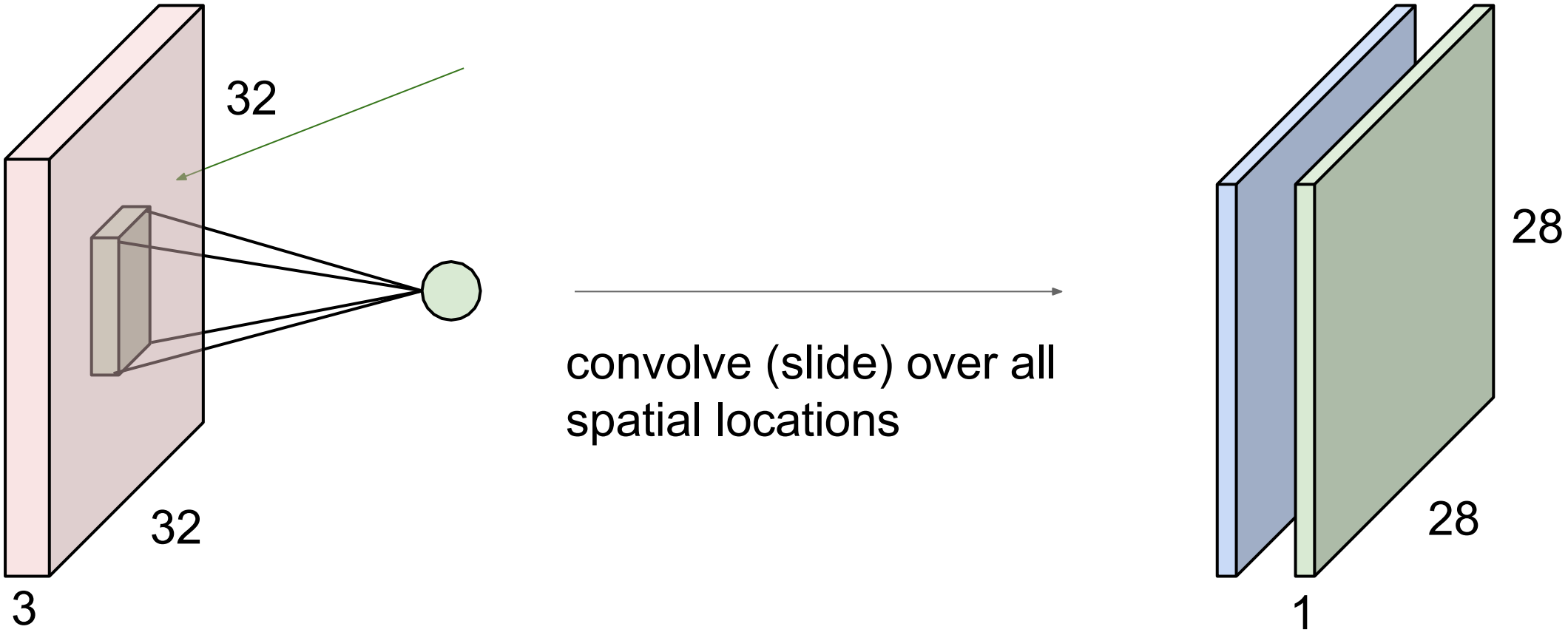


Convolution Layer

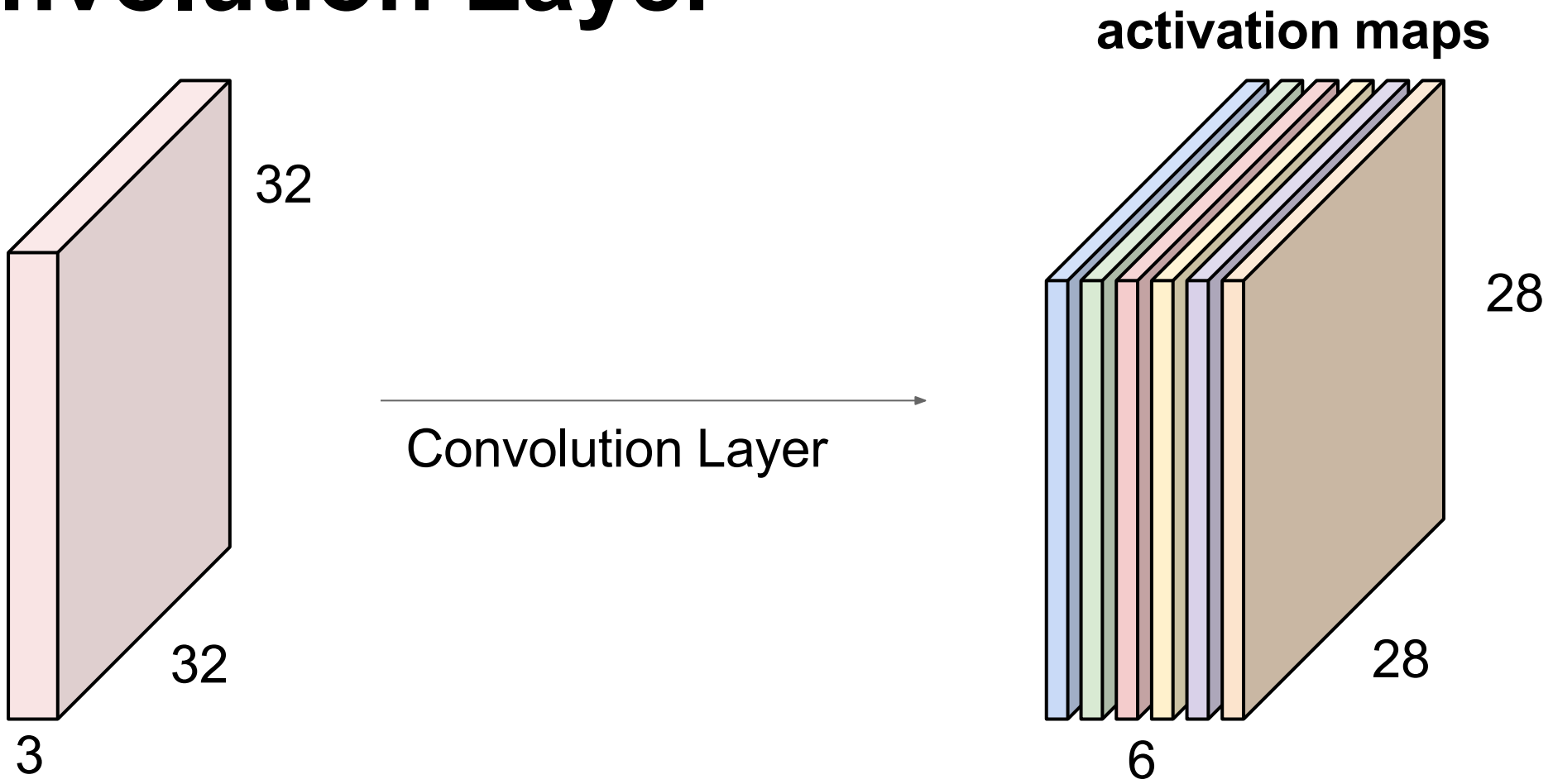


Convolution Layer

consider a second, green filter



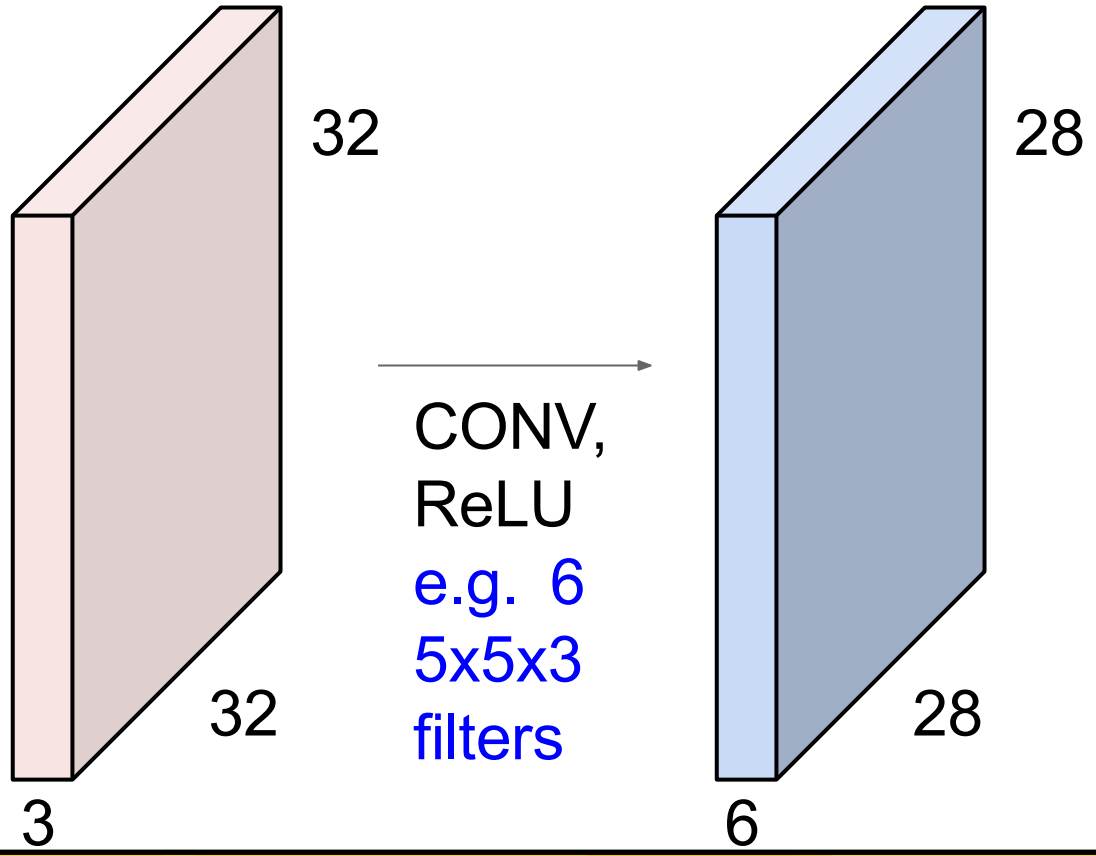
Convolution Layer



- For example, if we had 6 5x5x3 filters, we'll get 6 separate activation maps
- We stack these up to get a "new image" of size 28x28x6!

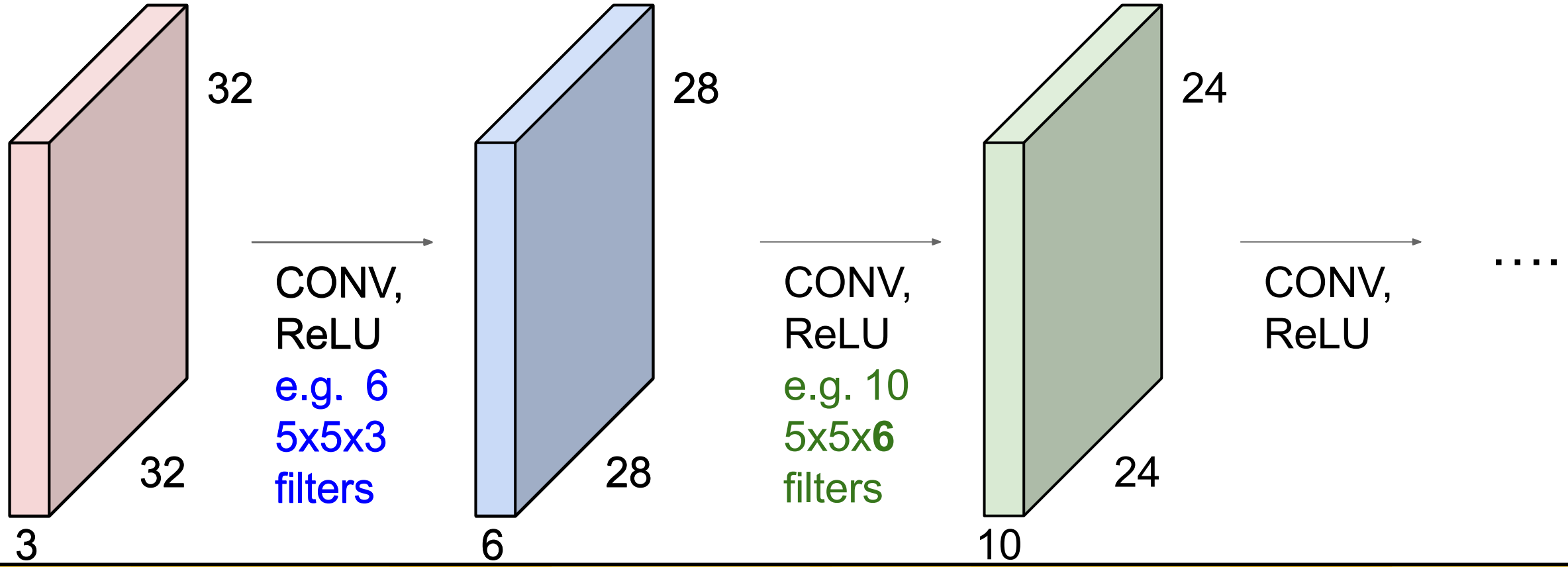
Convolution Layer

- **Preview:** ConvNet is a sequence of Convolution Layers, interspersed with activation functions

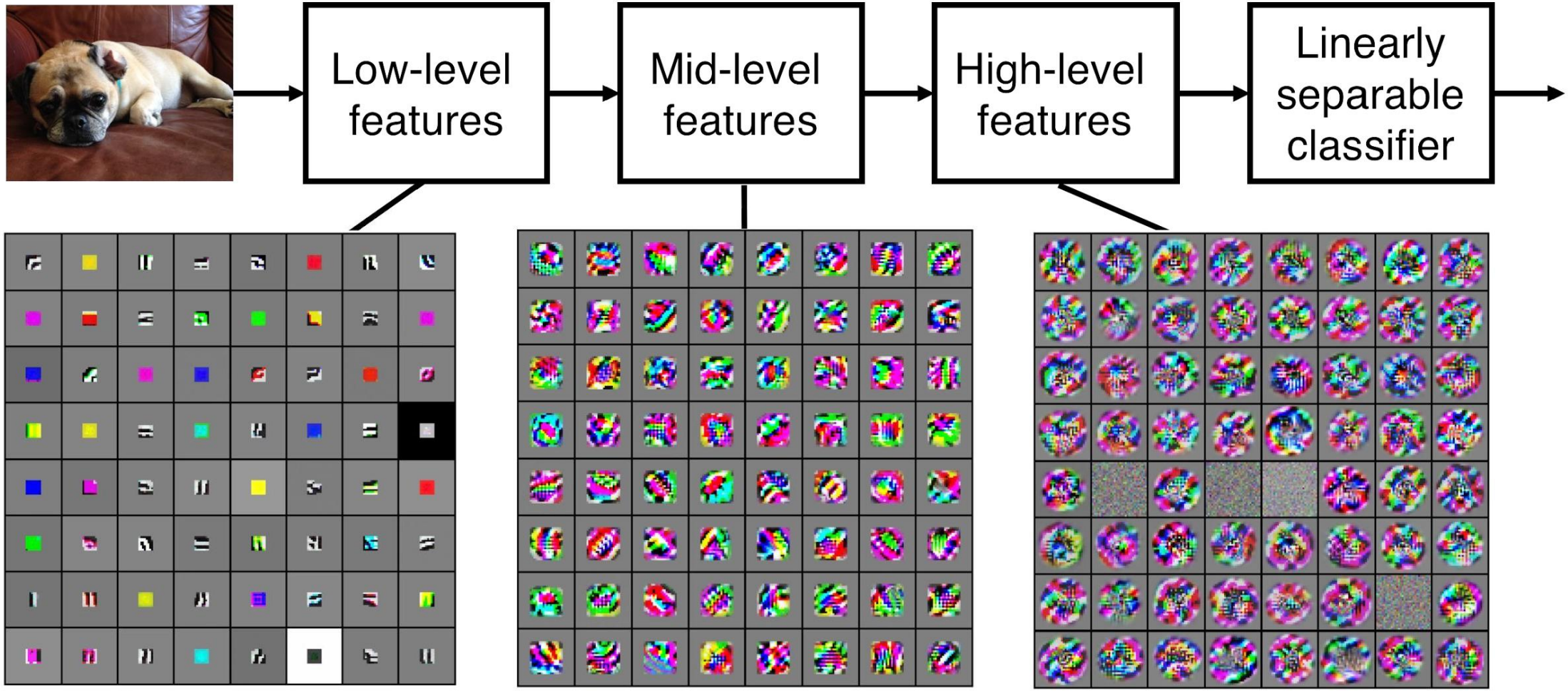


Convolution Layer

- **Preview:** ConvNet is a sequence of Convolution Layers, interspersed with activation functions



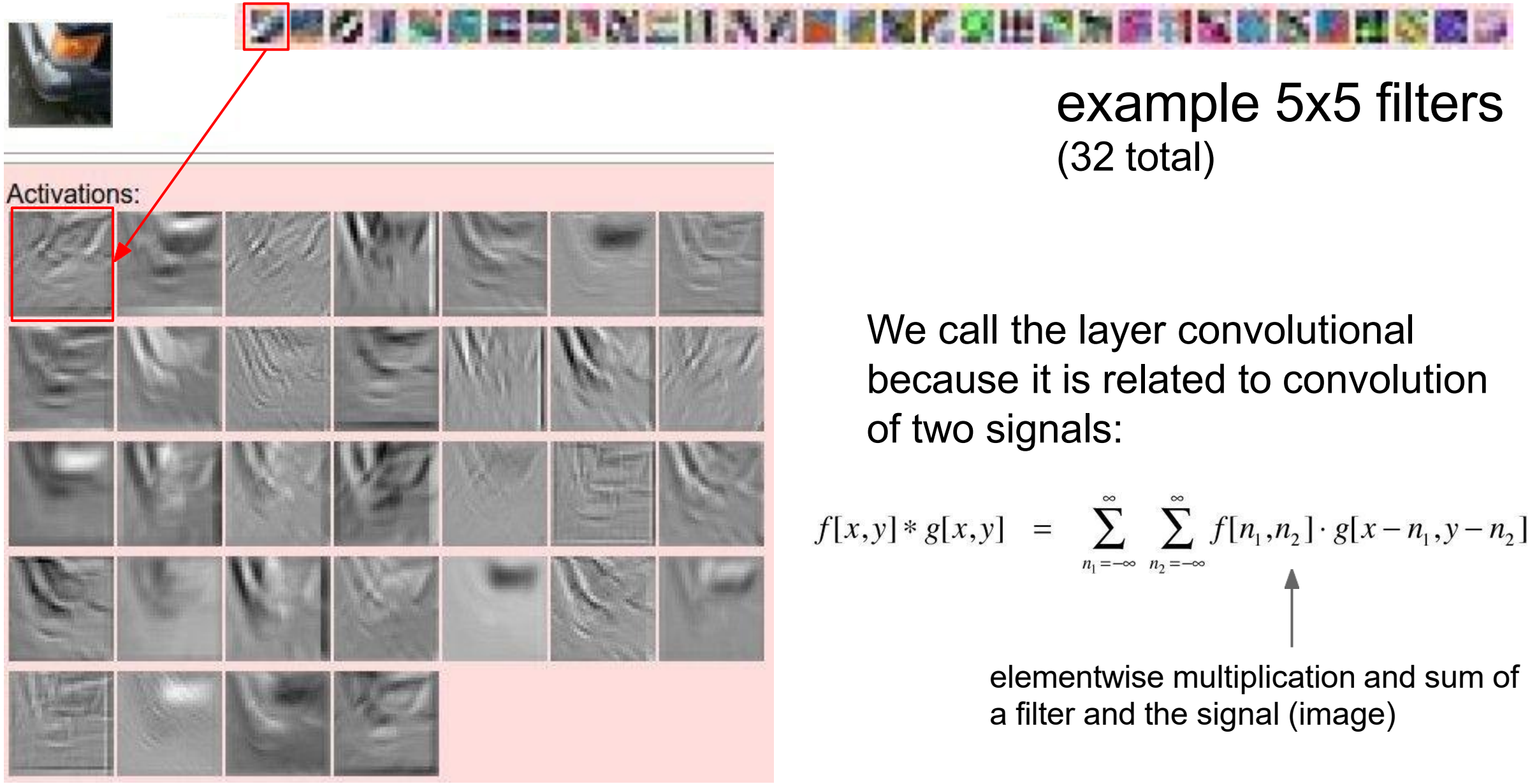
Feature extraction



VGG-16 Conv1_1

VGG-16 Conv3_2

VGG-16 Conv5_3

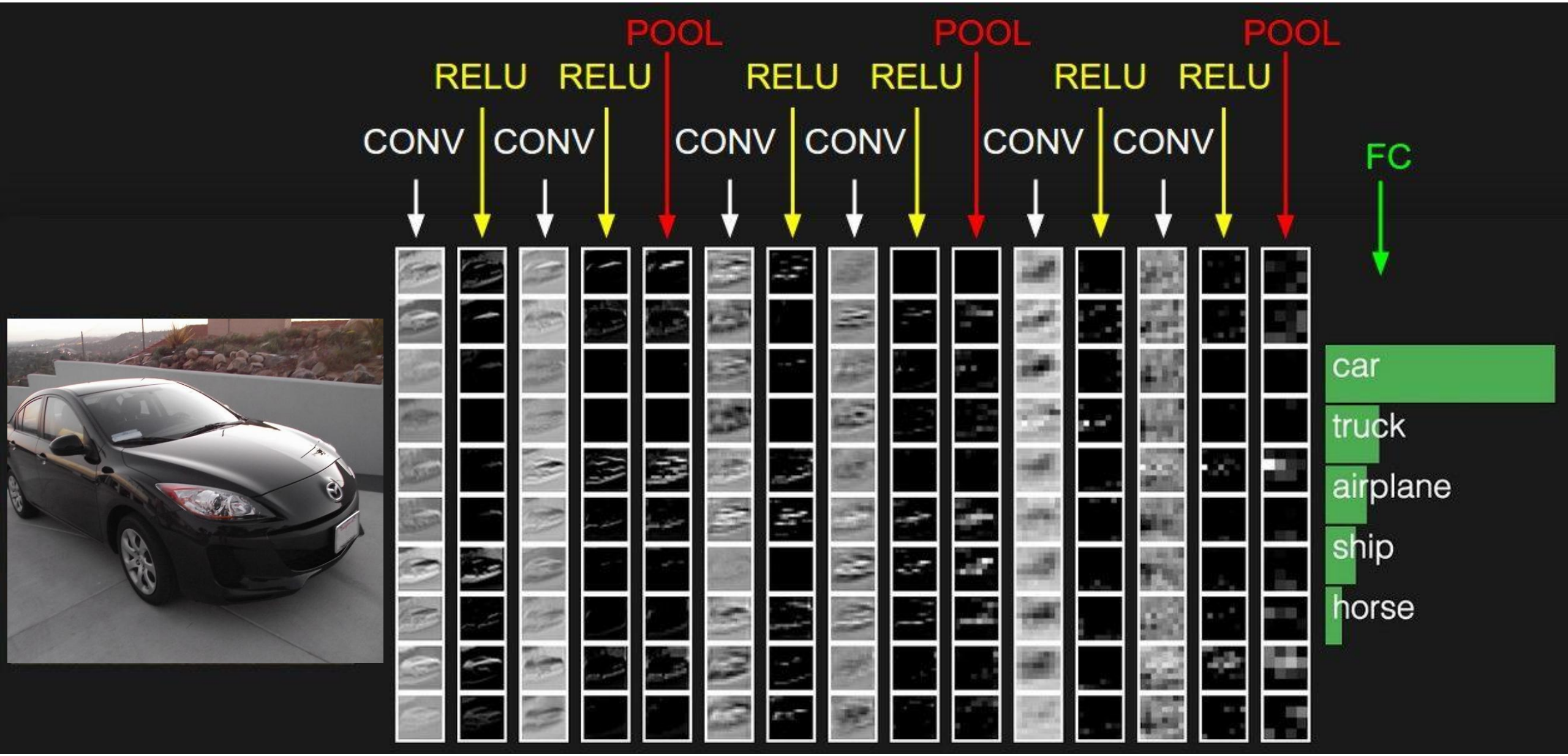


example 5x5 filters
(32 total)

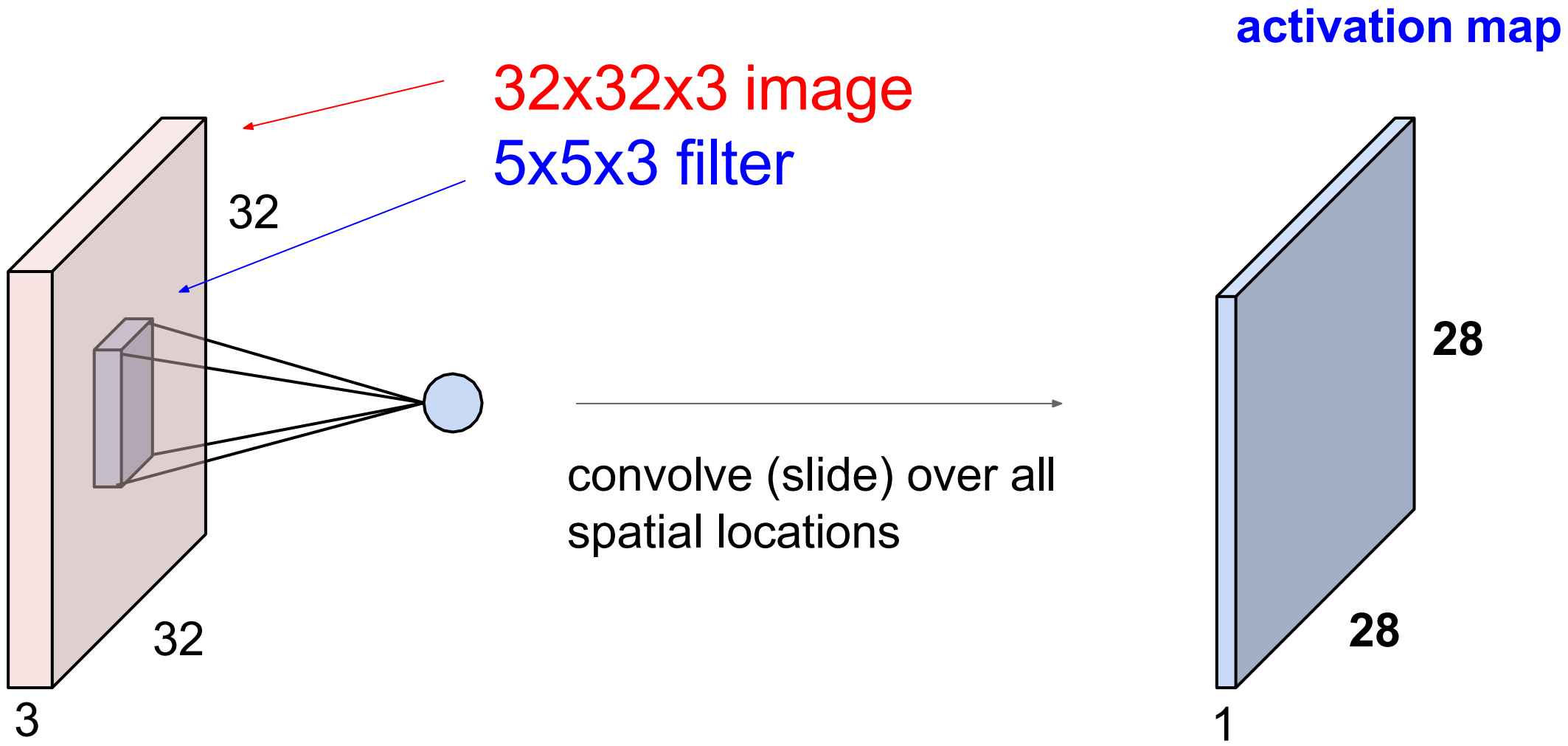
We call the layer convolutional because it is related to convolution of two signals:

$$f[x,y] * g[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2] \cdot g[x-n_1, y-n_2]$$

elementwise multiplication and sum of a filter and the signal (image)

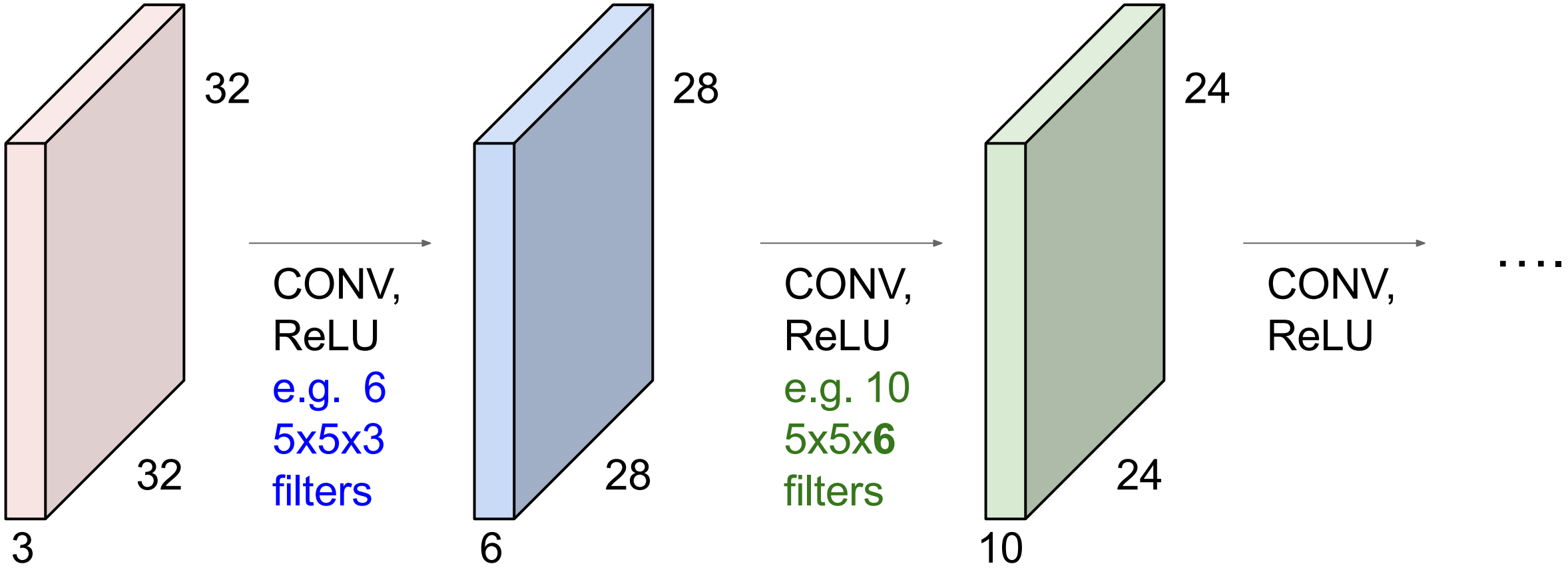

POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

Spatial dimensions



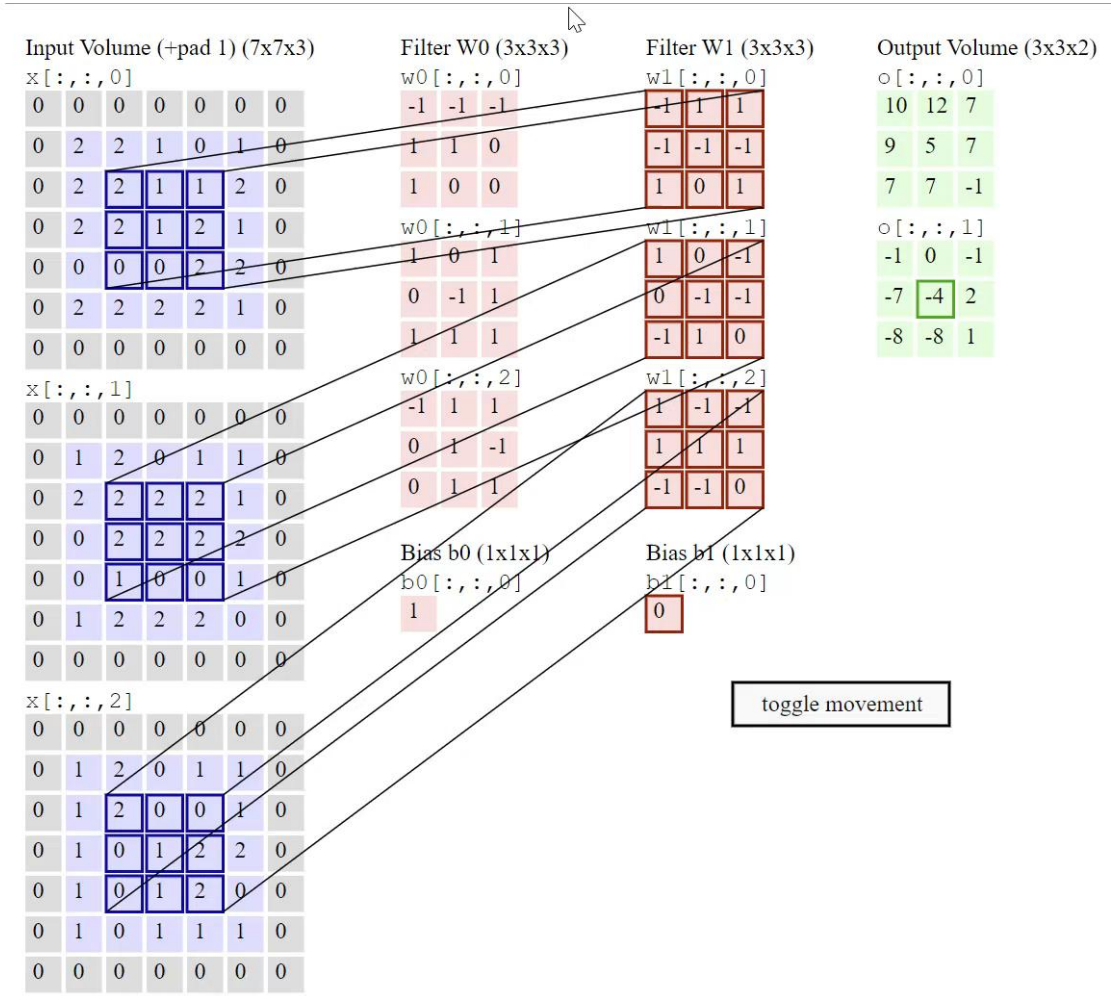
3 Layered CNN

- 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.

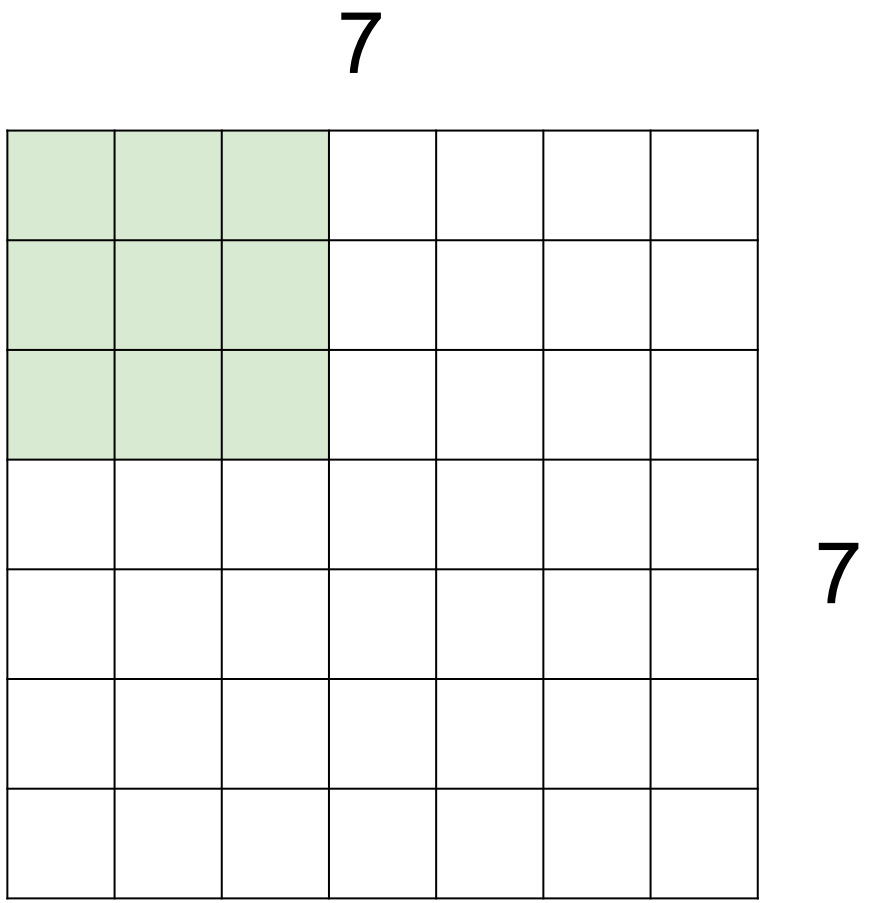


CNN

https://cs231n.github.io/convolutional-networks/

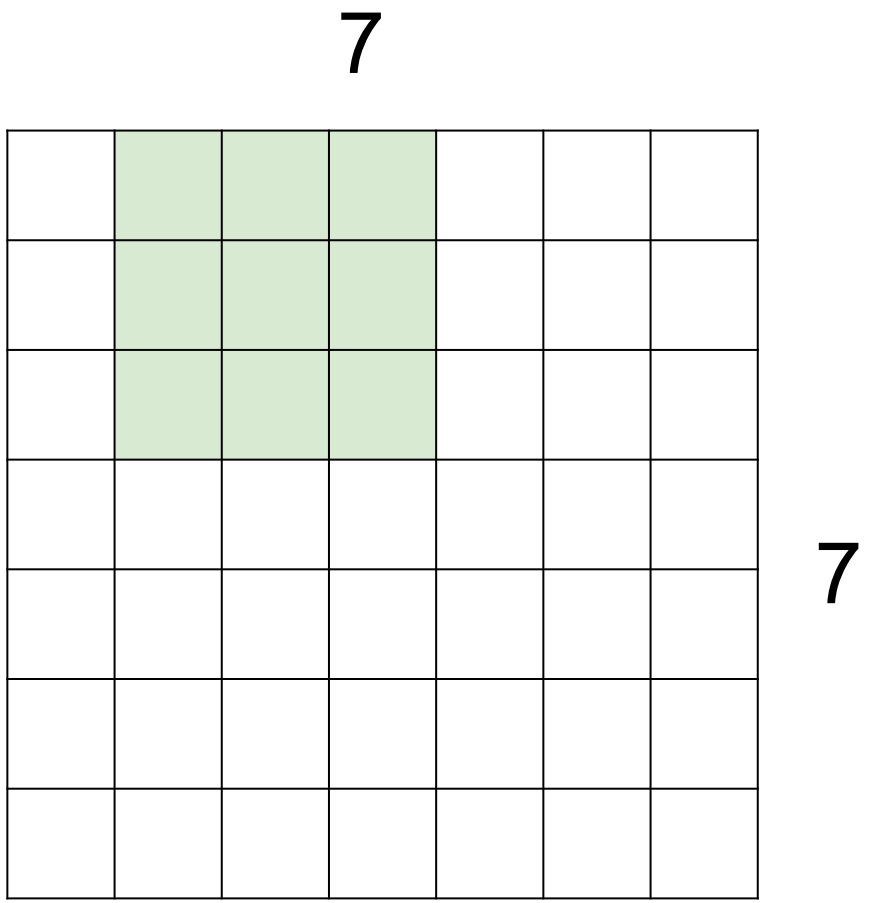


A closer look at spatial dimensions



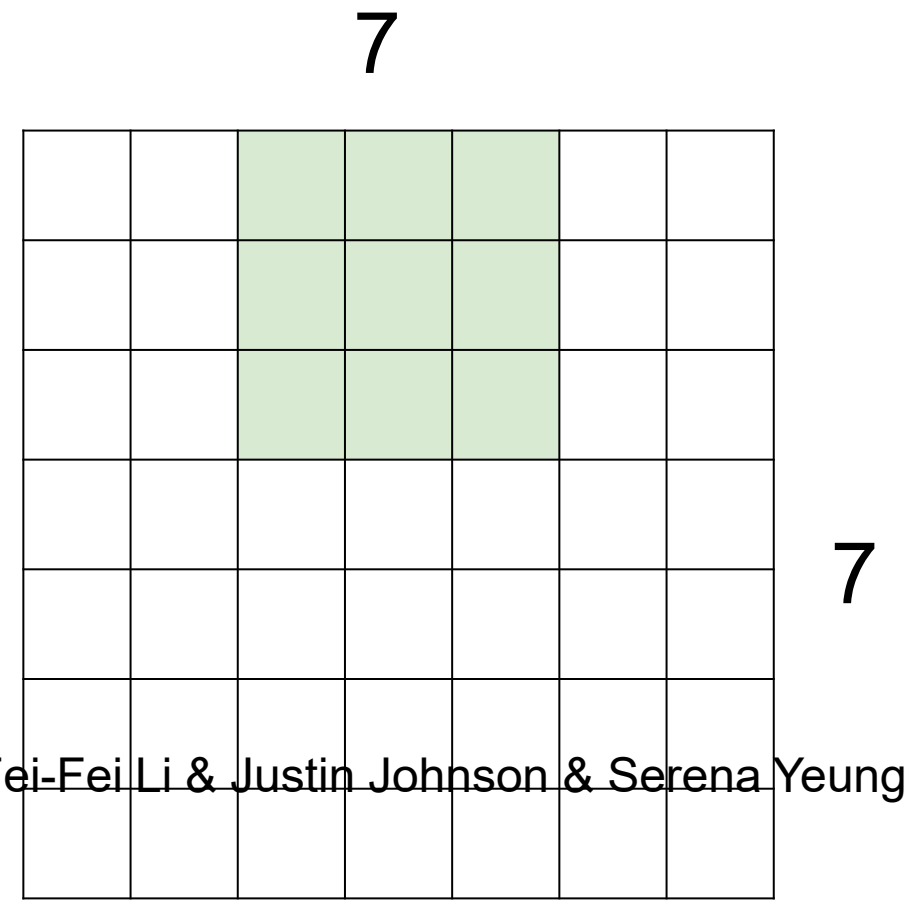
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions



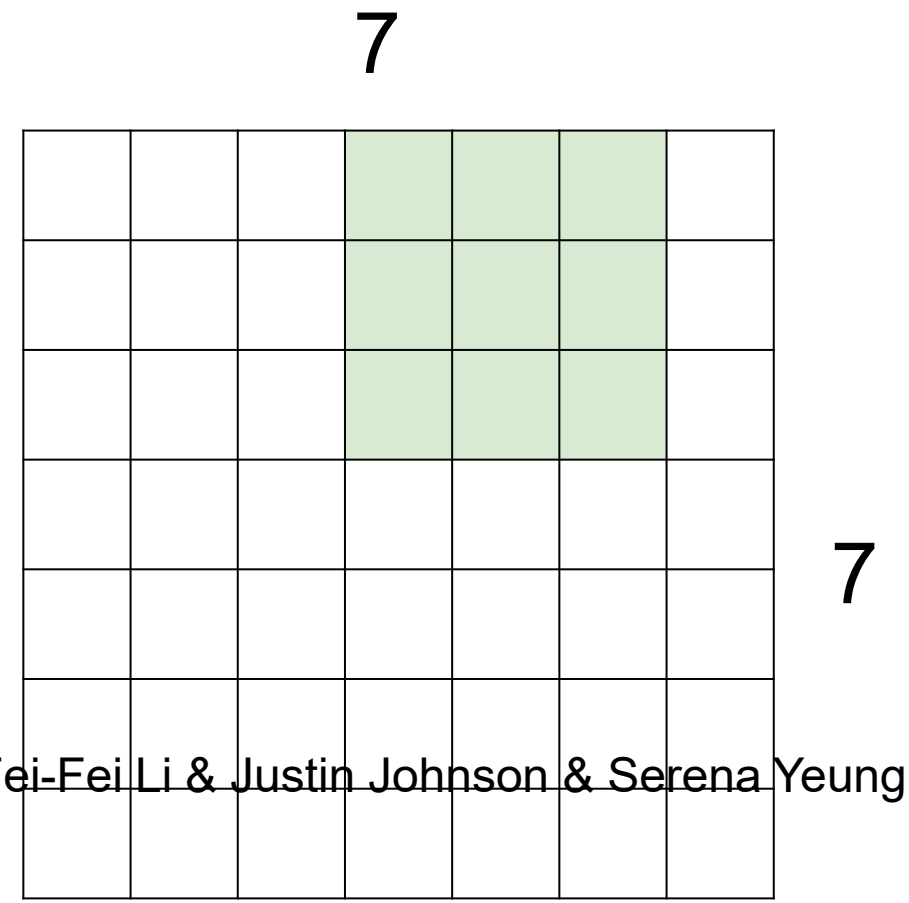
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions



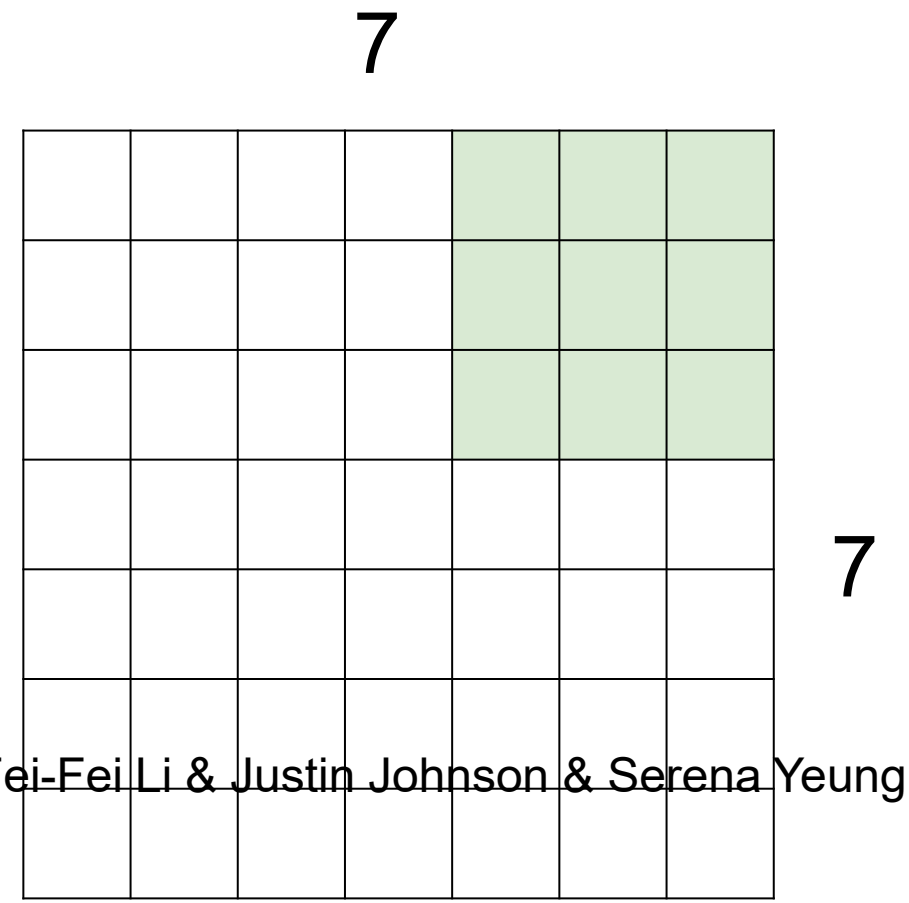
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions



7x7 input (spatially)
assume 3x3 filter

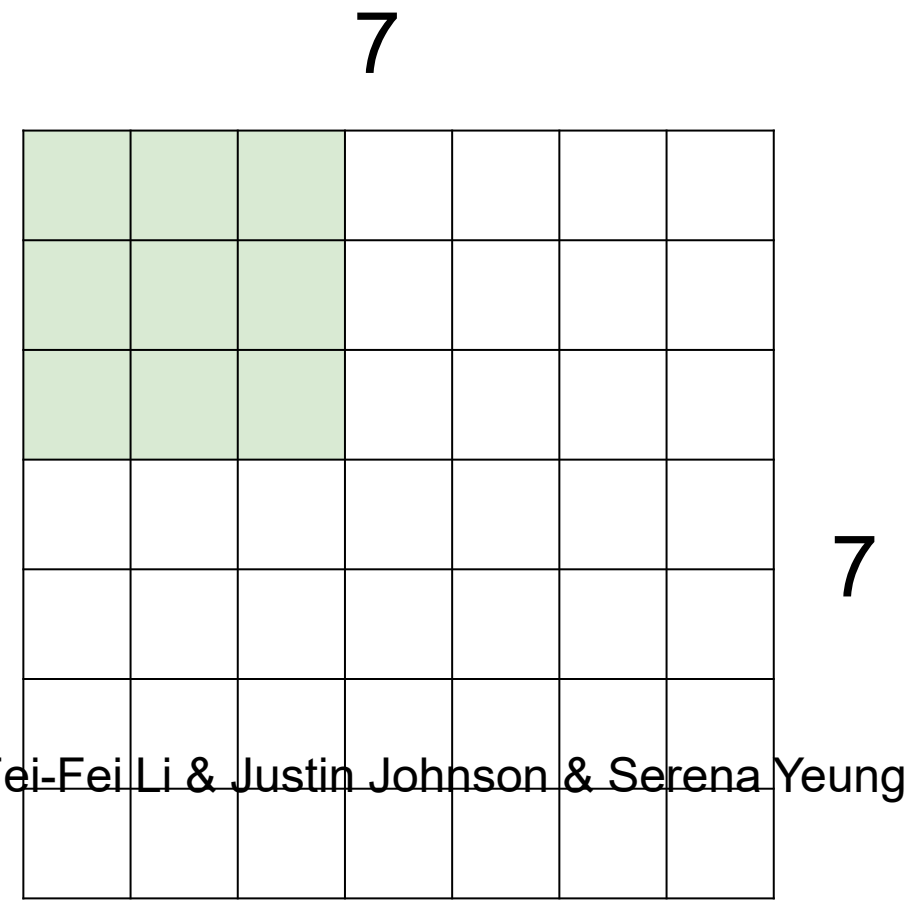
A closer look at spatial dimensions



7x7 input (spatially)
assume 3x3 filter

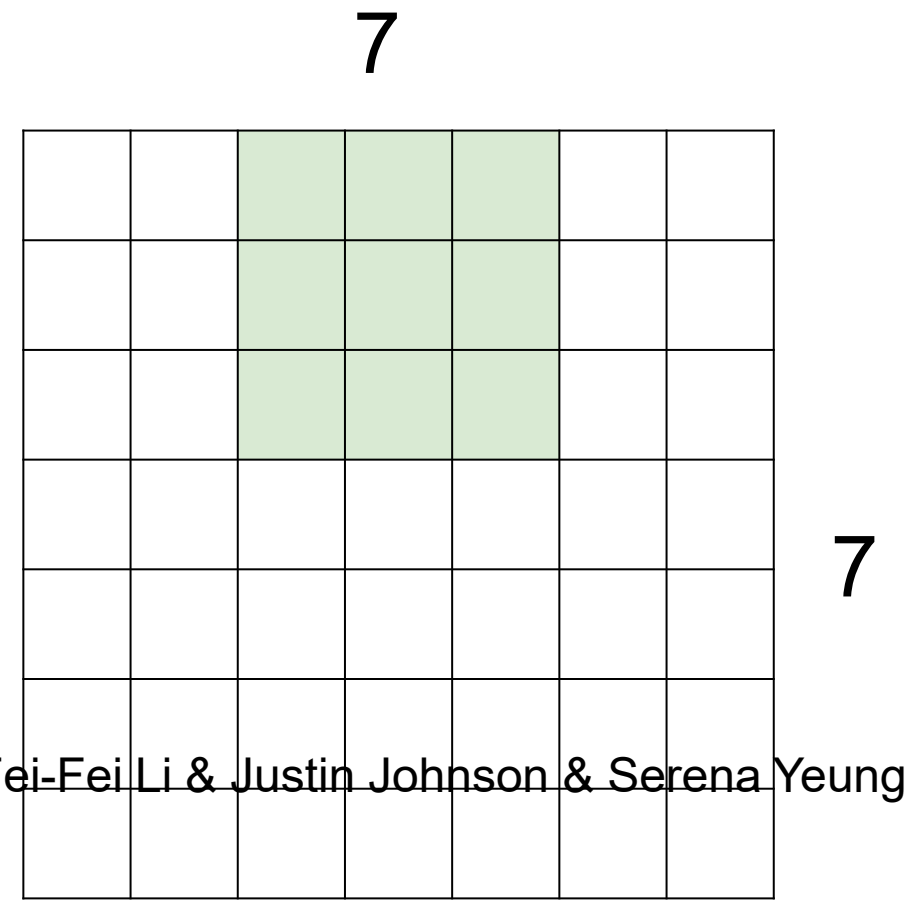
=> 5x5 output

A closer look at spatial dimensions



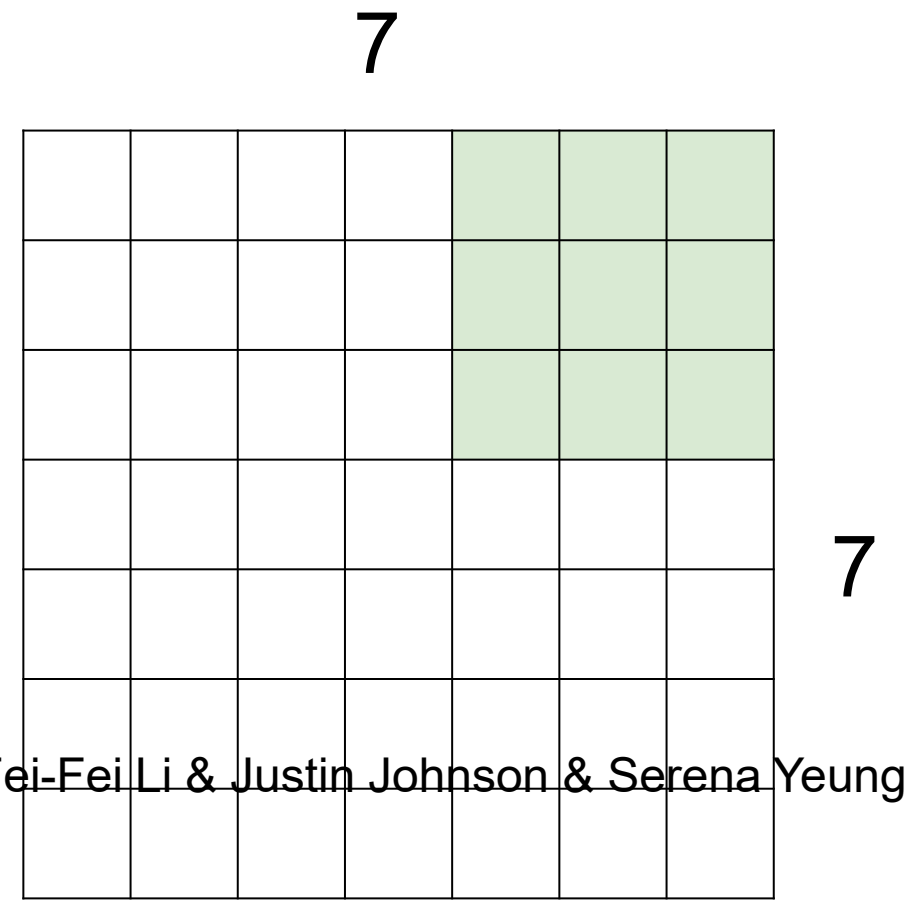
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions



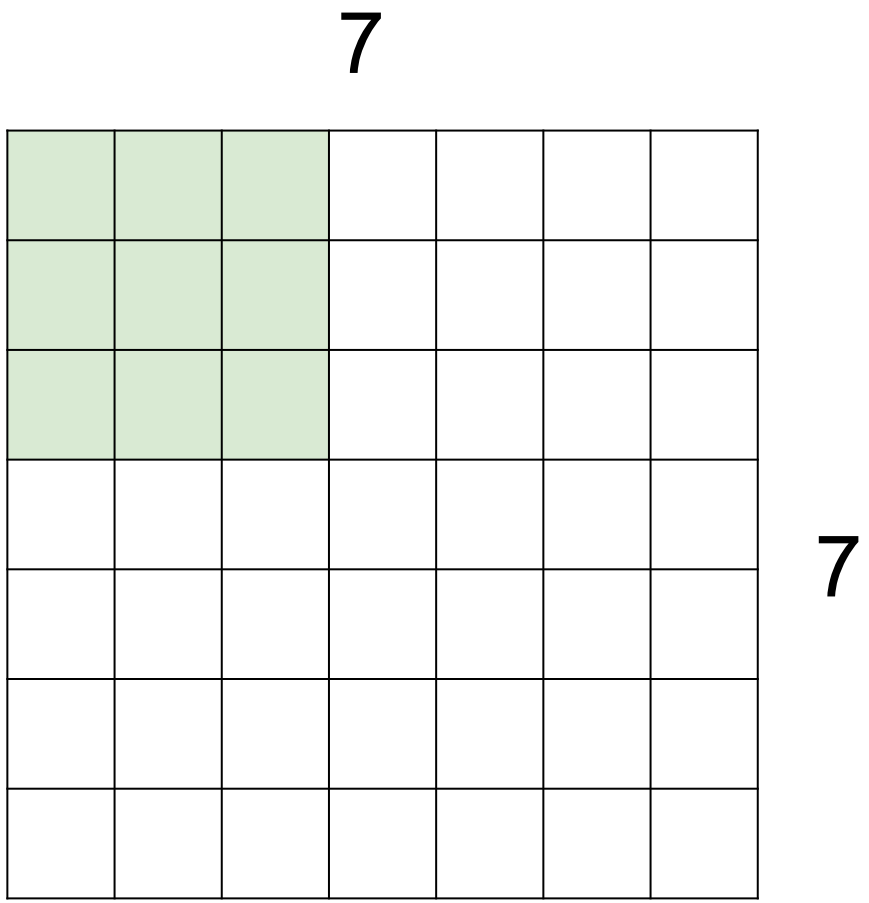
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions



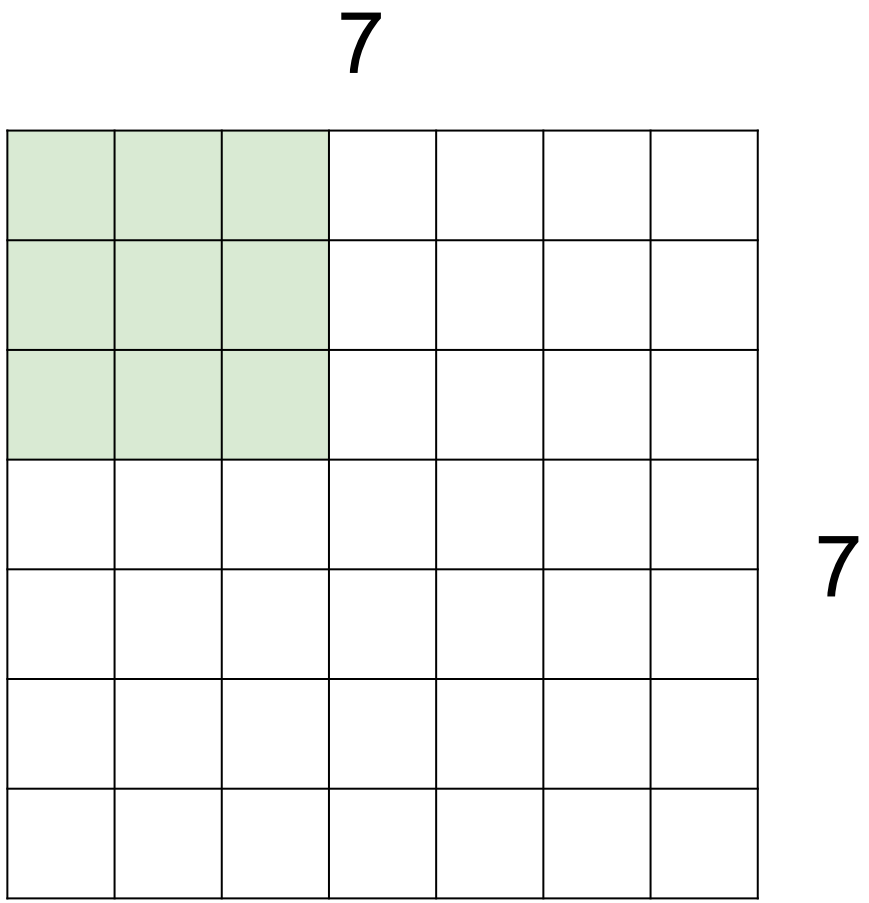
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

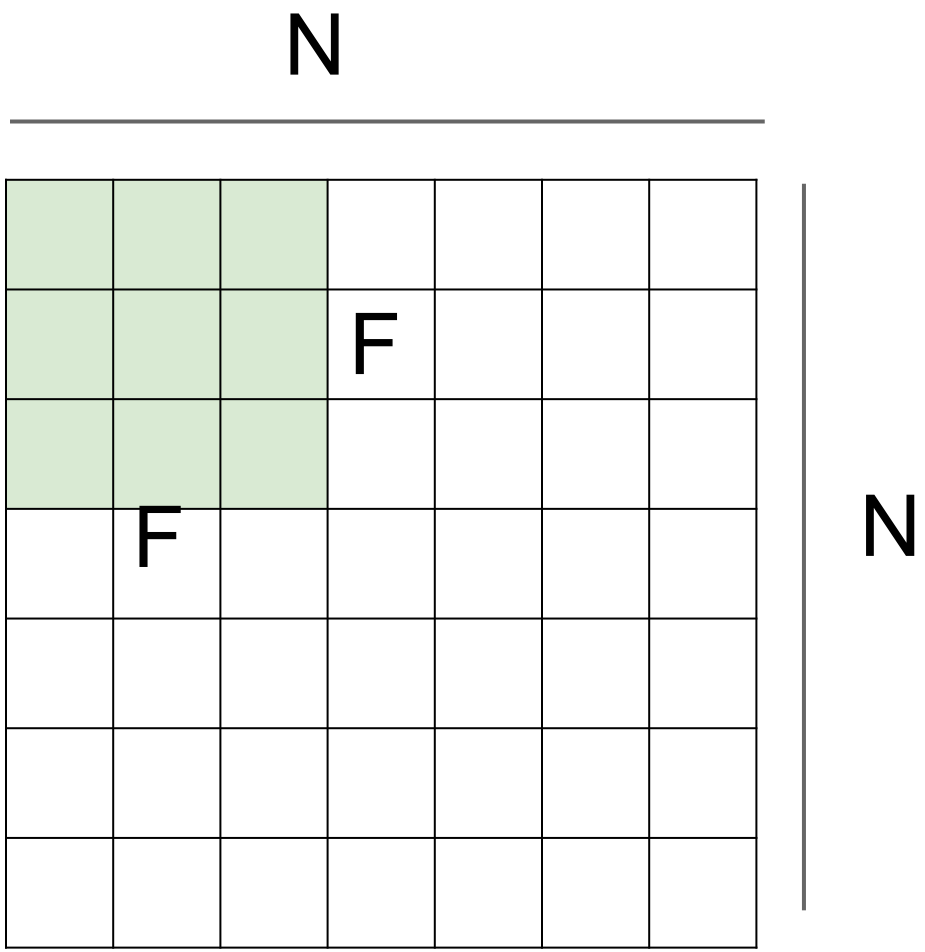
A closer look at spatial dimensions



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

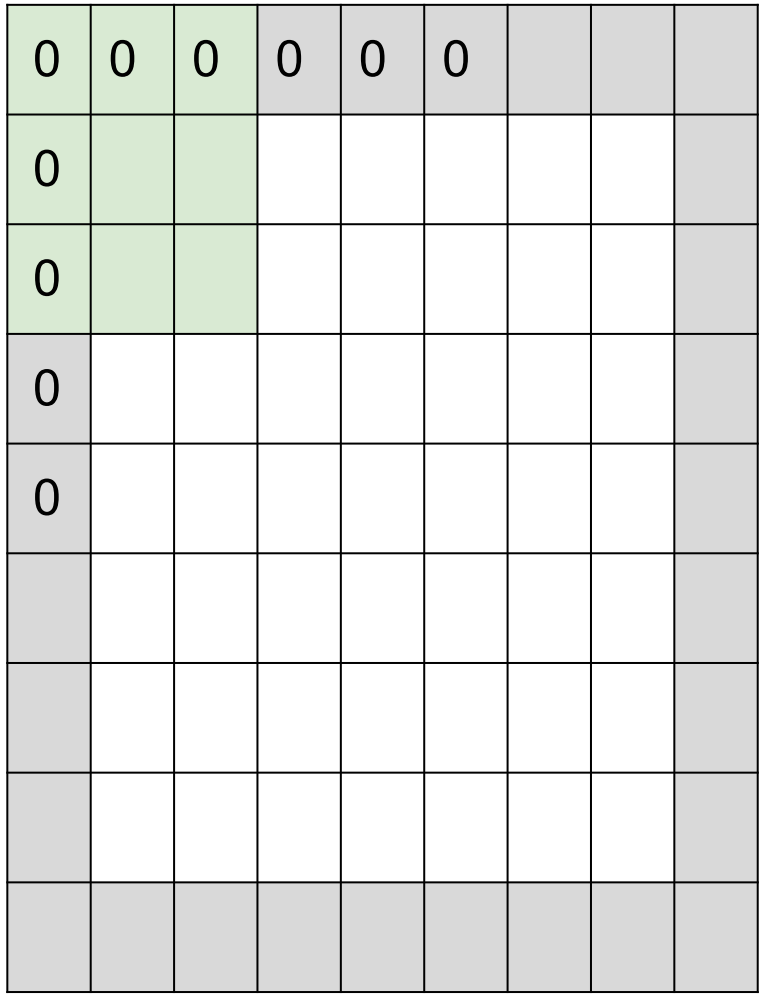
A closer look at spatial dimensions



Output size:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

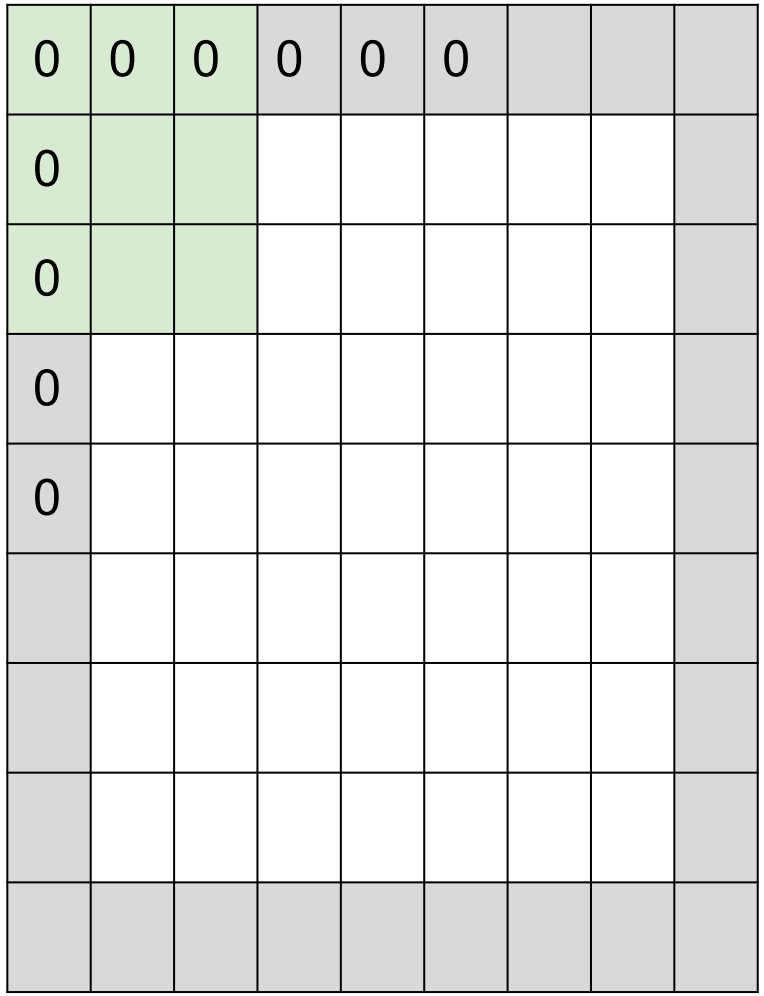
In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

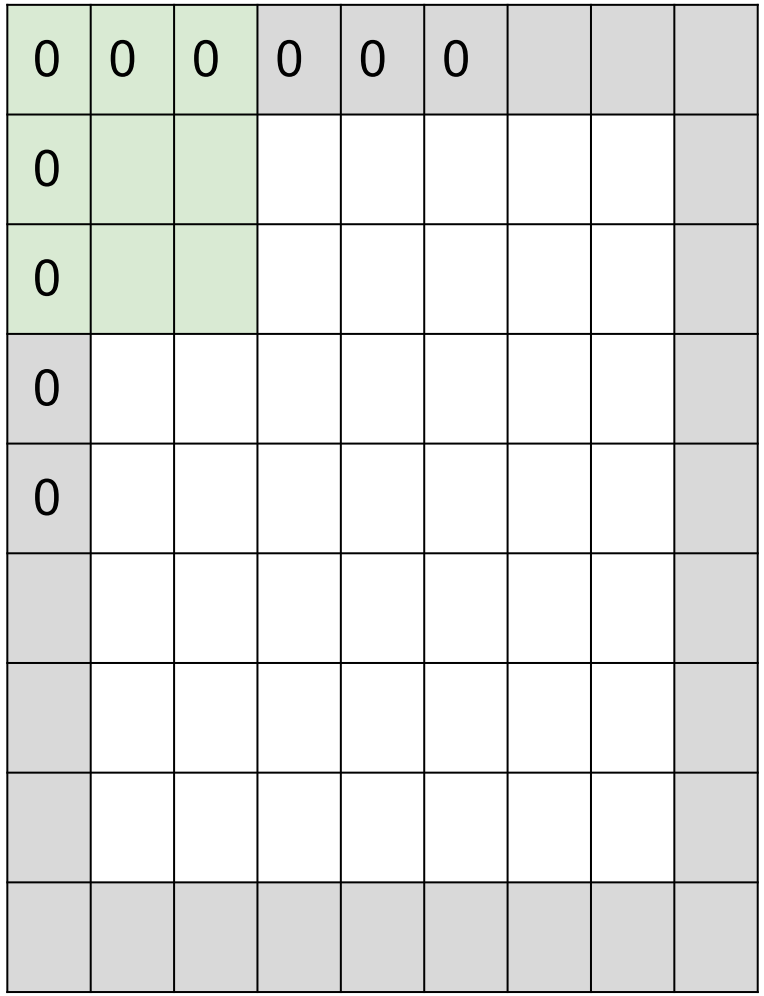
In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

In practice: Common to zero pad the border

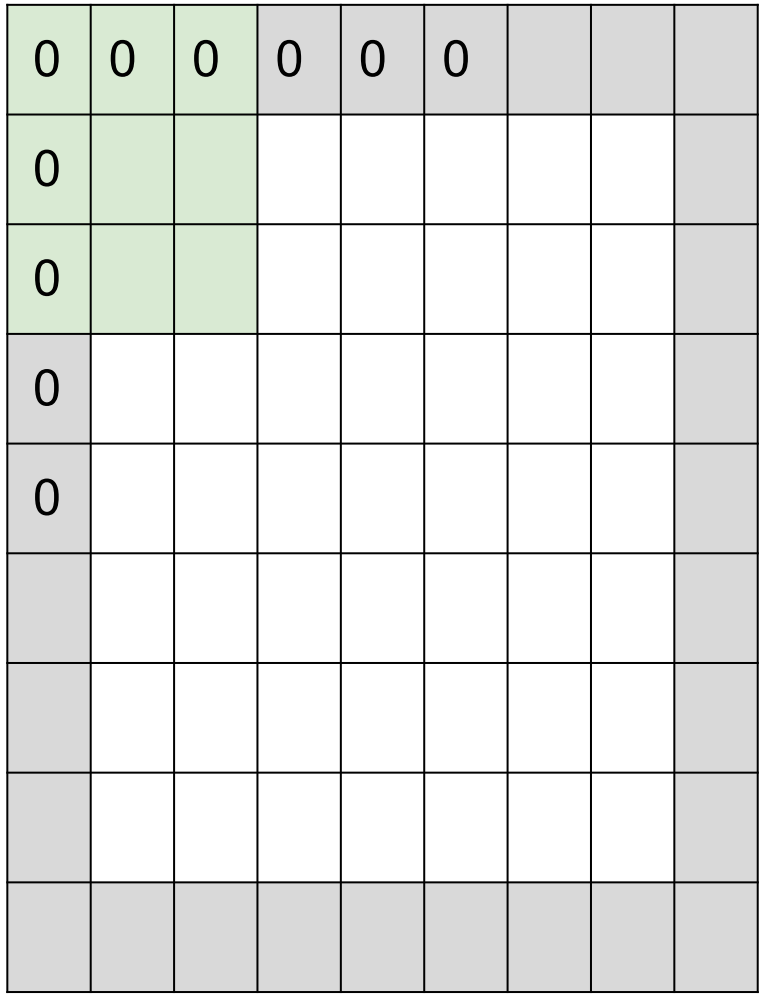


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

(recall:)
(N +2P - F) / stride + 1

In practice: Common to zero pad the border

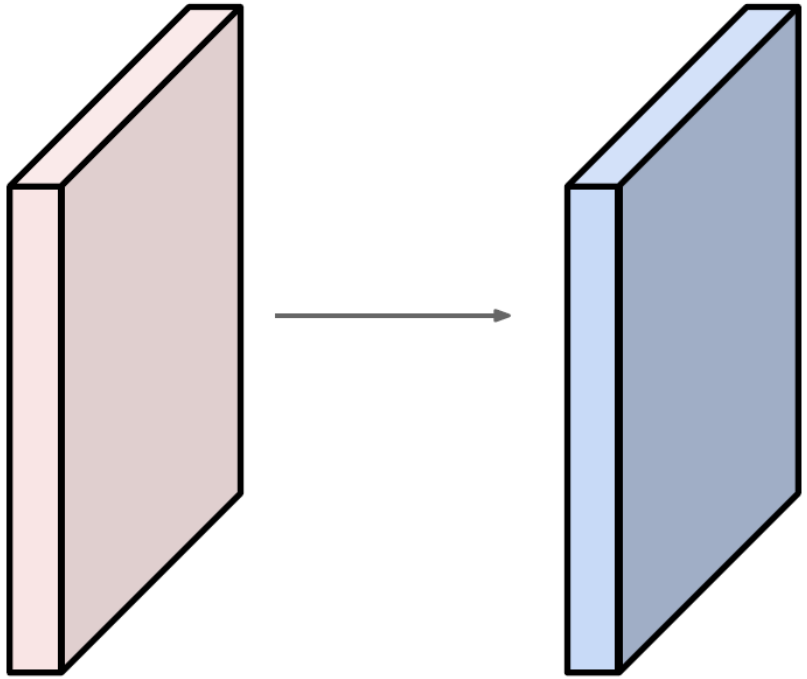


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

Exercise

- Input volume: **32x32x3**
- 10 5x5 filters with stride 1, pad 2

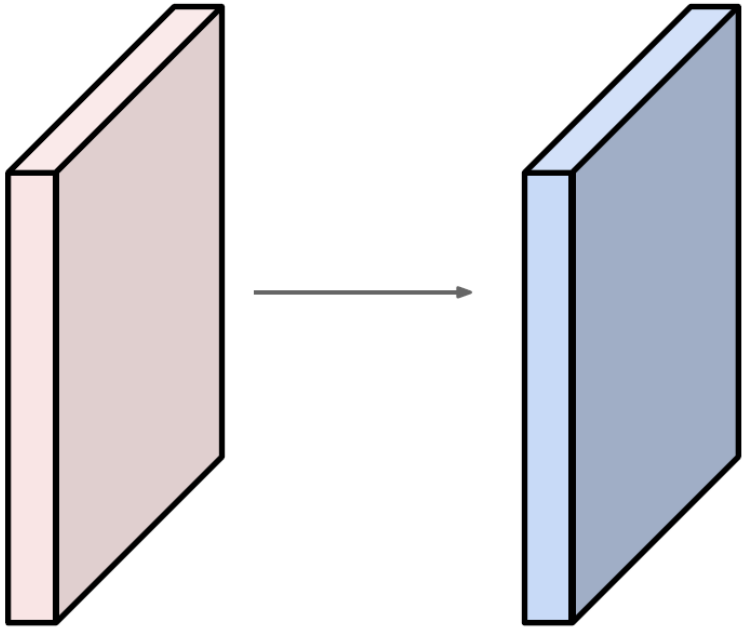


- Output volume size **?**
- (32+2*2-5)/1+1 = 32 spatially
- **32x32x10**

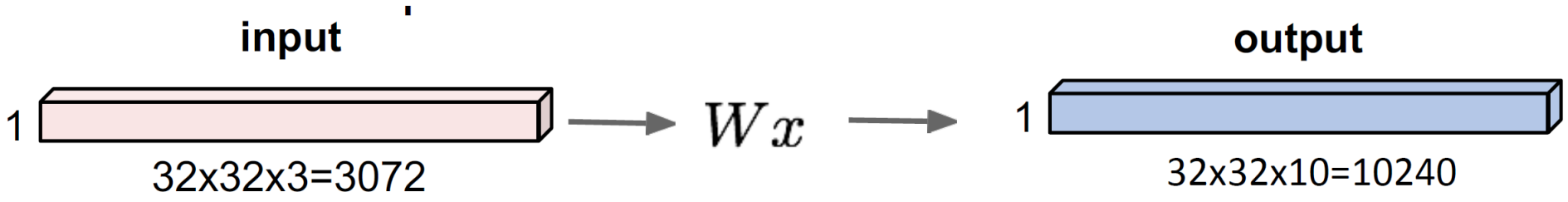
- Number of parameters in this layer **?**
- each filter has 5*5*3 + 1 = 76 params
- => 76*10 = **760**

Exercise

- Input volume: **32x32x3**
- Output volume: **32x32x10**



- Number of parameters if we use FC?



- (32x32x3+1) x (32x32x10)=**31,467,520**

Summary

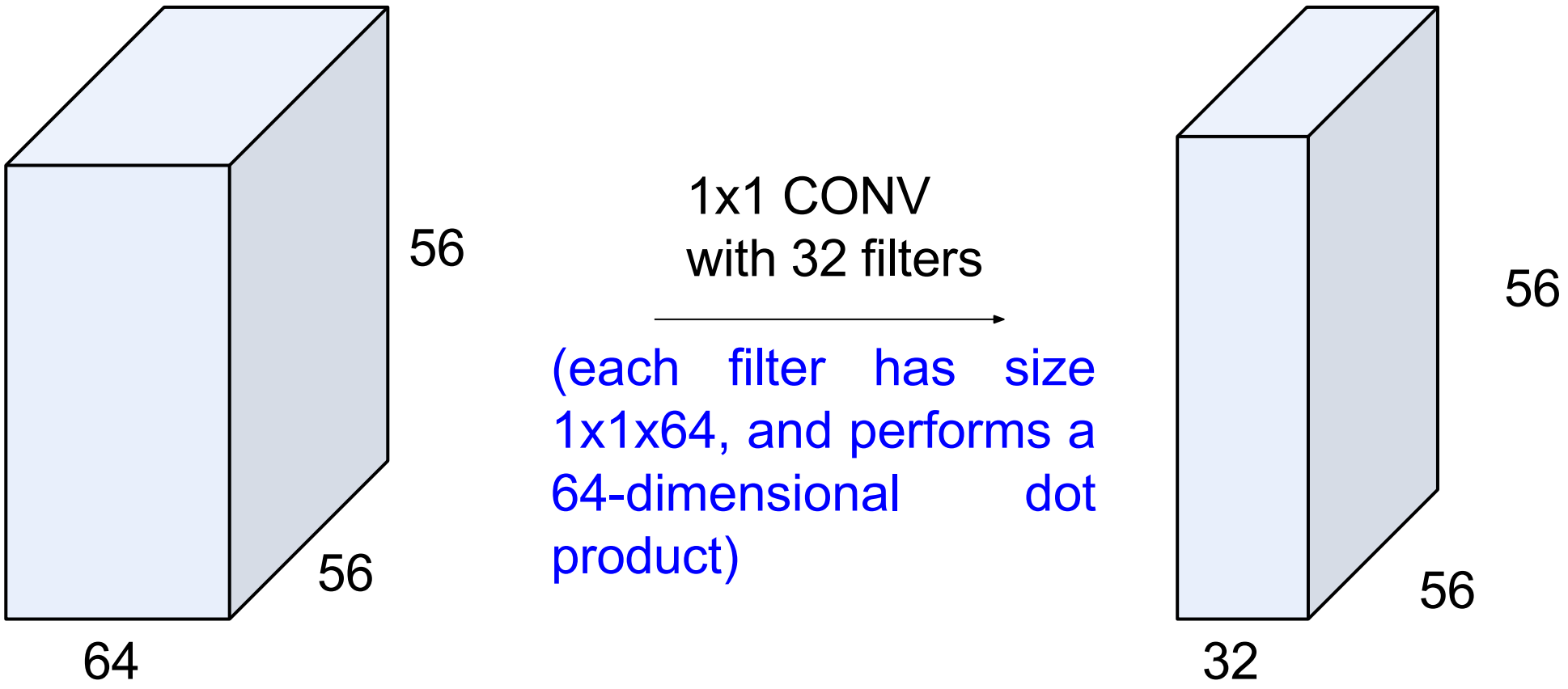
- Let's assume input is W x H x C
- Conv layer needs 4 hyperparameters:
  - Number of filters **K**
  - The filter size **F**
  - The stride **S**
  - The zero padding **P**
- This will produce an output W_2 x H_2 x K where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$
  - Number of parameters: F_2CK and K biases

Common settings:

K = (powers of 2, e.g. 32, 64, 128, 512)

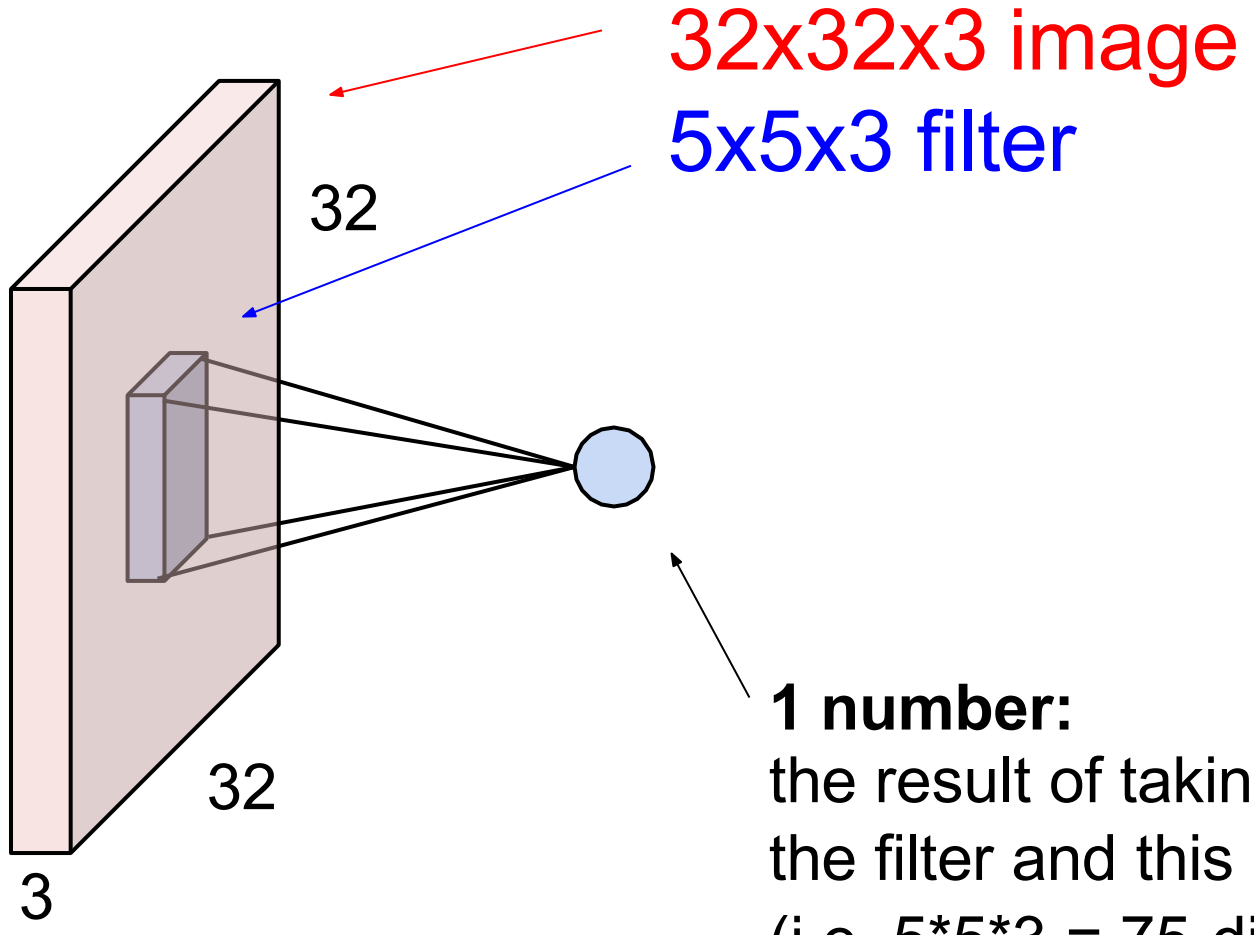
- F = 3, S = 1, P = 1
- F = 5, S = 1, P = 2
- F = 5, S = 2, P = ? (whatever fits)
- F = 1, S = 1, P = 0

1 x 1 Convolution Layer



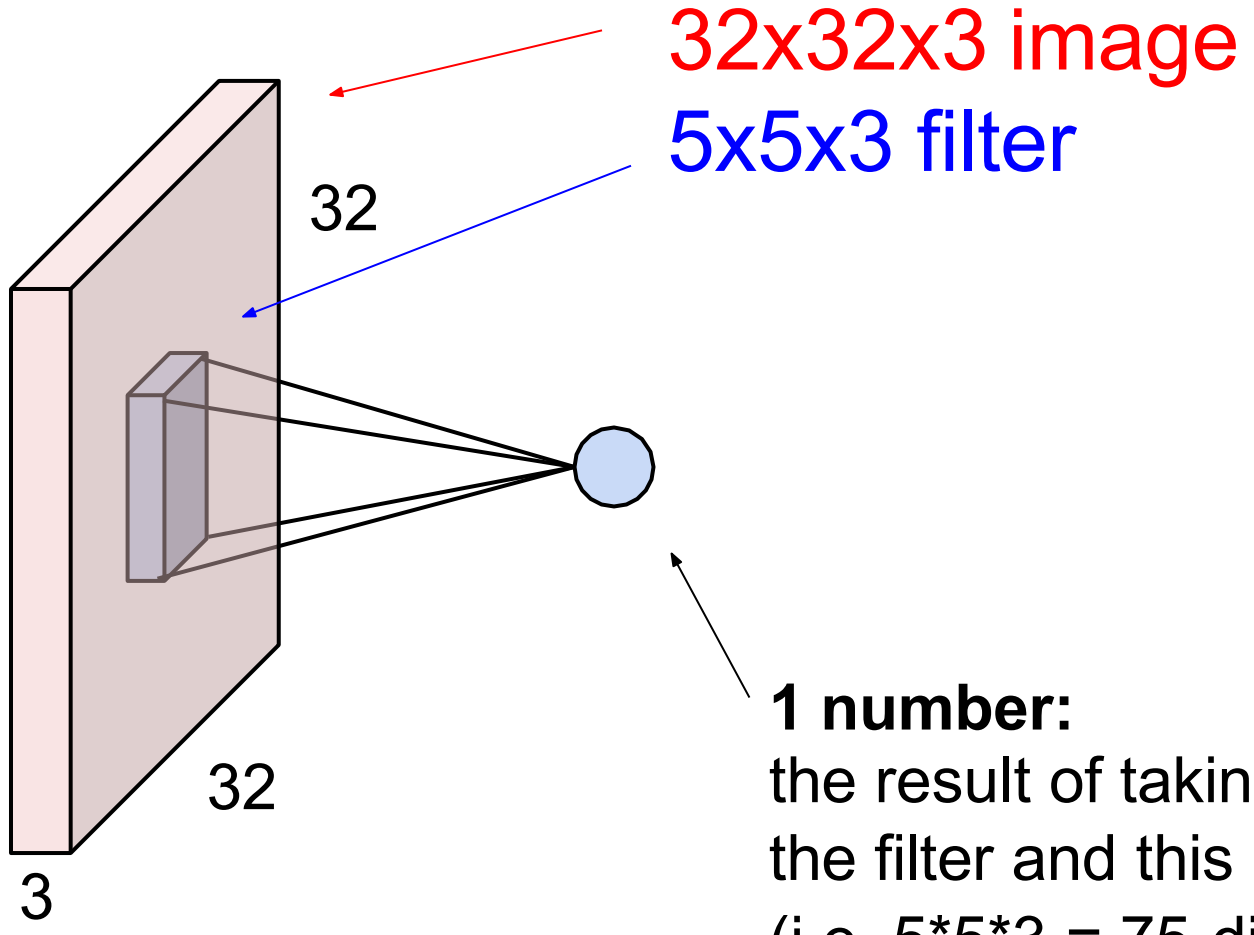
- Dimensionality reduction in filter dimension space

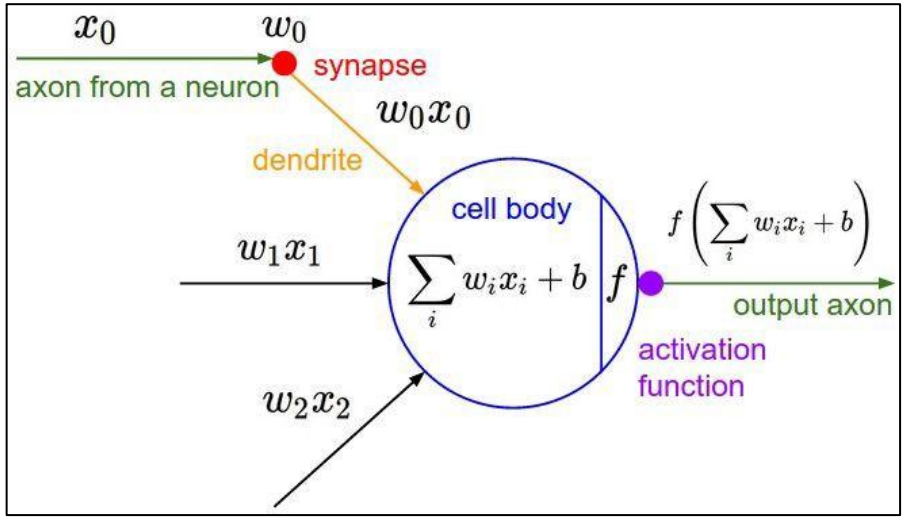
The brain/neuron view of CONV Layer



1 number:
the result of taking a dot product between the filter and this part of the image
(i.e. 5*5*3 = 75-dimensional dot product)

The brain/neuron view of CONV Layer

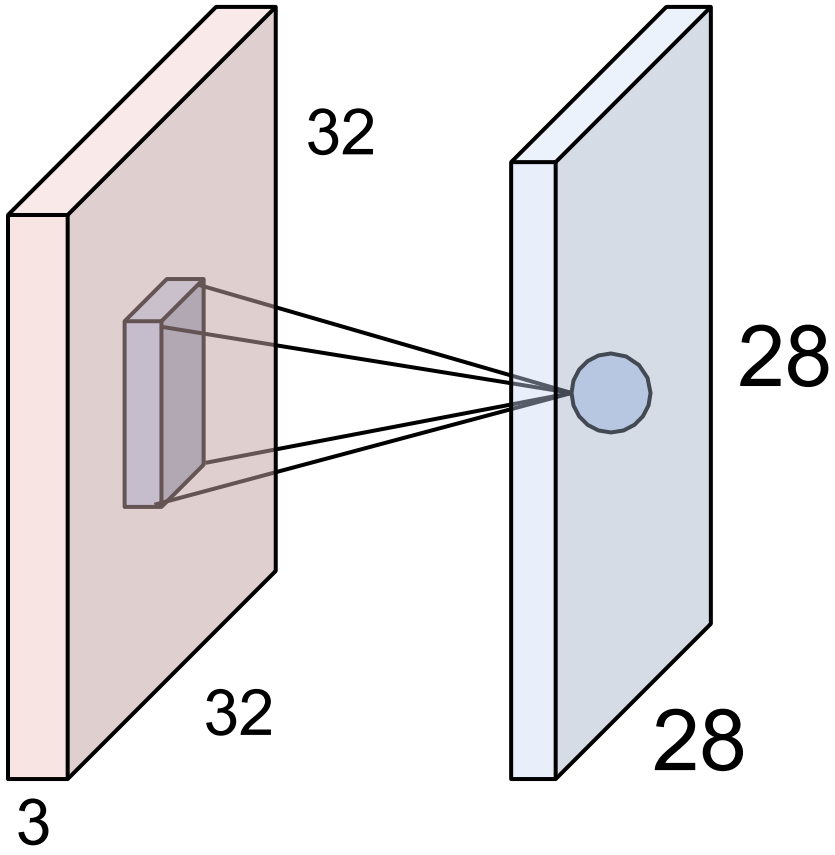


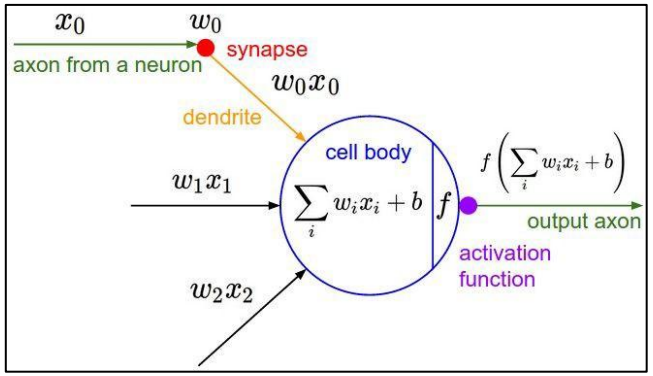


It's just a neuron with local connectivity...

1 number:
the result of taking a dot product between the filter and this part of the image
(i.e. 5*5*3 = 75-dimensional dot product)

The brain/neuron view of CONV Layer



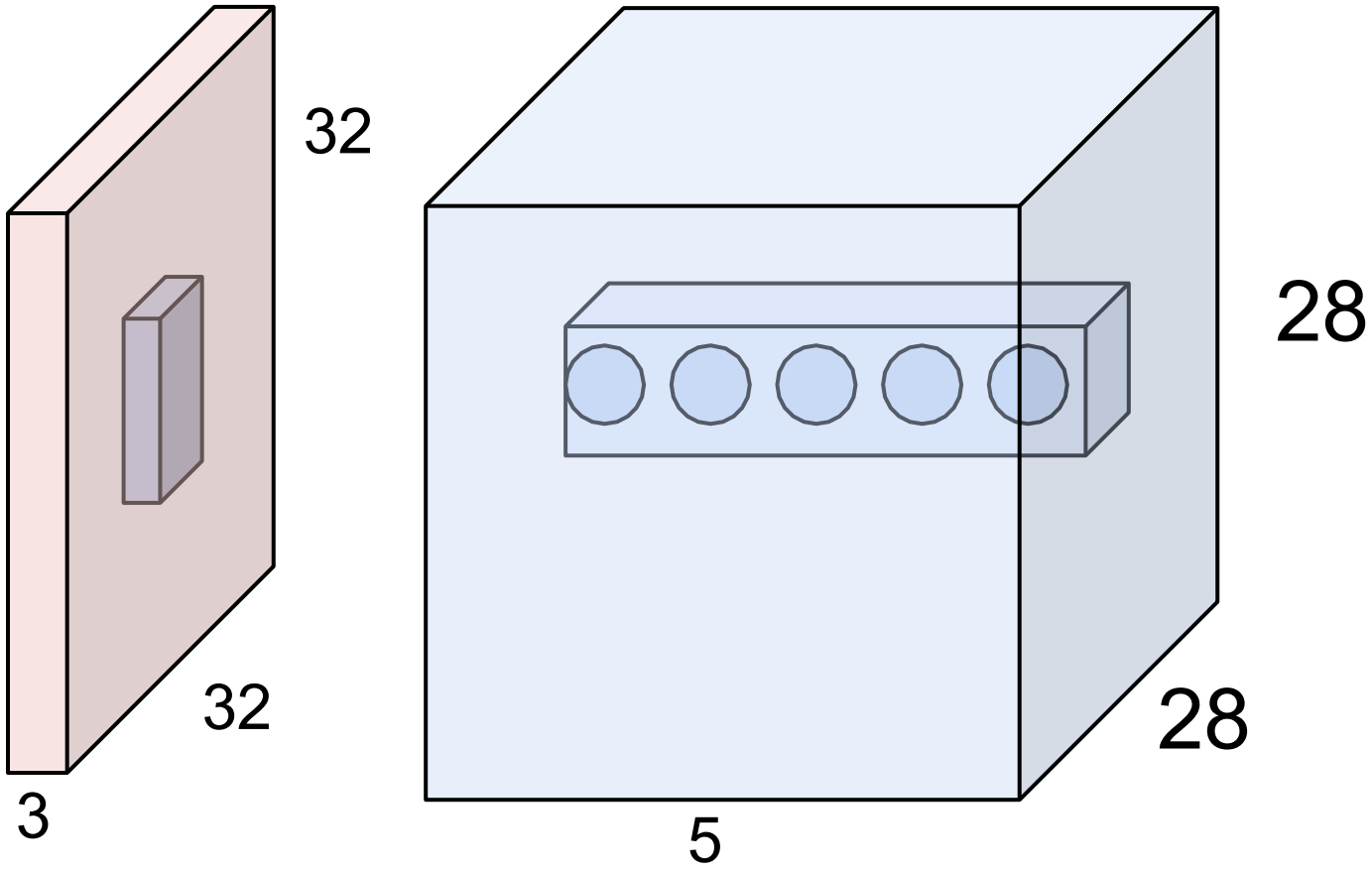


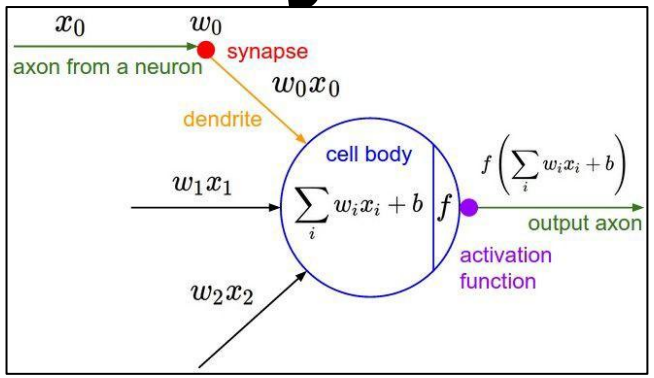
An activation map is a 28x28 sheet of neuron outputs:

1. Each is connected to a small region in the input
2. All of them share parameters

“5x5 filter” -> “5x5 receptive field for each neuron”

The brain/neuron view of CONV Layer





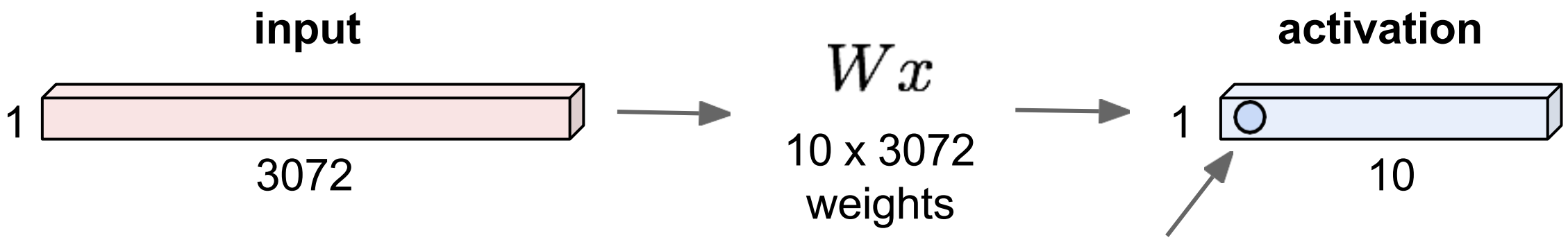
E.g. with 5 filters,
CONV layer consists of neurons arranged in a 3D grid (28x28x5)

There will be 5 different neurons all looking at the same region in the input volume

CONV vs. Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1

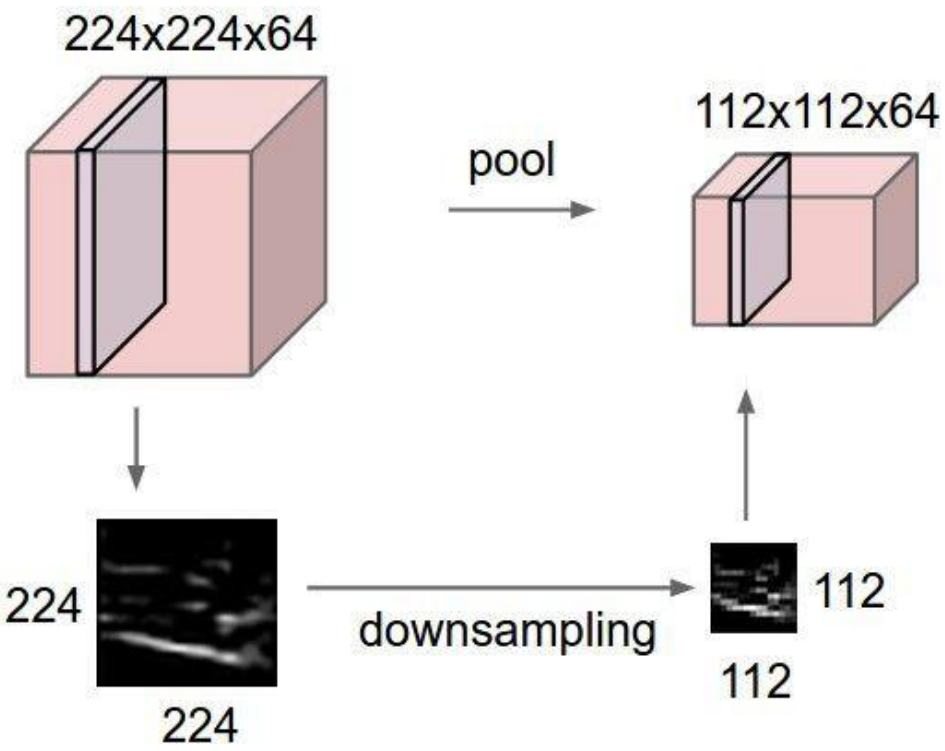
Each neuron looks at the full input volume



1 number:
the result of taking a dot product between a row of W and the input (a 3072-dimensional dot product)

Pooling

Pooling



- makes the representations smaller and more manageable
- reduces the spatial dimension
- operates over each activation map independently

Max Pooling

Single depth slice

x

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

y

max pool with 2x2 filters
and stride 2

6	8
3	4

Summary

- Let's assume input is W x H x C
- Max Pooling layer needs 2 hyperparameters:
  - The spatial extent **F**
  - The stride **S**
- This will produce an output W_2 x H_2 x K where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - Number of parameters: 0

Common settings:

F = 2, S = 2
F = 3, S = 2

Summary

- Applications of convolutional neural networks
- Convolution/Pooling layer
- Techniques in CNN
  - Data preprocessing, Batch normalization, Early stopping, Regularization, Dropout

References

- Stanford CS231n